

18 CHỦ ĐỀ 9+ ÔN THI TỐT NGHIỆP MOBILE



Tài liệu luyện thi hiệu quả
Hệ thống kiến thức trọng tâm

0) Gu ra đề & chiến lược làm bài	4
Bắt hay gặp (đánh dấu đỏ khi học)	5
Bảng quy đổi nhanh Kotlin ↔ Java (để đọc đề)	6
1.1 Application Components (Building Blocks)	8
1.2 AndroidManifest.xml	9
1.3 Intent & Navigation	10
1.4 Permissions (runtime)	11
1.5 Android Stack & Architecture	12
1.6 Launch Modes & Context	12
2) Lifecycle & State: Activity/Fragment – onSaveInstanceState	13
2.1 Activity Lifecycle (flow)	14
2.2 onSaveInstanceState vs onPause/onResume	15
2.3 Fragment essentials	16
Checklist "Bắt" cho Mục 2:	17
3) UI: Layouts – Adapter/ListView – WebView – Toast/Dialog	19
3.1 MVC (mức khái niệm)	20
3.2 Layout selection (Linear/Relative/Constraint)	21
3.3 Views & event handling	22
3.4 ListView & Adapter & RecyclerView	22
3.5 WebView	24
3.6 Toast & Dialog	26
3.7 Toast vs Snackbar vs Dialog vs BottomSheet	26
3.8 Pickers: DatePicker, TimePicker, NumberPicker (đảm bảo valid format).	27
3.9 Custom UI: Các bước tạo Custom Toast và Custom Dialog (dùng setView).	27
3.10 BottomSheet – Khái niệm & Phân loại	27
3.11 Đơn vị đo lường:	27
3.12 View Lifecycle & View Hierarchy	28
4) Resources & Local Storage: strings/fonts/preferences/files	29
4.1 Resources & Localization	30
4.2 Fonts, Files, SharedPreferences	31
4.3 Internal vs External Storage (Lưu trữ file)	32
4.4 Fonts (API Level)	32
4.5 XML Processing (SAX vs DOM)	32
4.6 SQLite Types & SharedPrefs Path	32
5) Data Persistence: XML/JSON – SQLite/Cursor – Room – Realm	33
5.1 XML & JSON	34
5.2 SQLite & Cursor	35
5.3 Room	36
5.4 Realm	37
6) Network & Cloud: REST – Firebase Auth – RTDB/Storage – Firestore compare	38
6.1 Web services & REST	39
6.2 Firebase Auth	41
6.3 Realtime Database + Firestore	41
6.4 Cloud Storage (Quản lý file)	43

7) Background & Notifications: Threading – Service – Broadcast – Local/FCM	43
7.1 Multi-threading basics:	44
7.2 Services	46
7.3 BroadcastReceiver	47
7.4 Notifications	47
7.5 Firebase Cloud Messaging (FCM - Remote)	48
7.6 Foreground Service & AsyncTask Cancel	49
Trang cuối: Mini checklist trước khi đi thi	50
PHỤ LỤC A — TRẮC NGHIỆM “LẠ” / TRIVIA (KIỂU: LỊCH SỬ, AI SÁNG TẠO, NĂM NÀO)	51
KOTLIN	54
🛡️ NHÓM 1: CƠ CHẾ HỆ THỐNG & KIẾN TRÚC	55
📐 NHÓM 2: LAYOUT & CONSTRAINTLAYOUT	55
📋 NHÓM 3: ADAPTER & RECYCLERVIEW (RV)	56
💾 NHÓM 4: STORAGE & RESOURCES	57
🔧 NHÓM 5: SERIALIZATION & REALM	57
🌐 NHÓM 6: WEBVIEW & JAVASCRIPT	58
📄 NHÓM 7: KHÁC (XML, FIREBASE)	59

0) Gu ra đề & chiến lược làm bài

Gu ra đề (từ đề Kỹ thuật lập trình + đề tự luận mobile cũ)

- Trắc nghiệm bao phủ rộng, nhiều câu tiểu tiết; ưu tiên hiểu hành vi (behavior) hơn là viết code dài.
- Hay hỏi: so sánh A vs B; tình huống lifecycle/flow; cái gì bắt buộc (API level/Manifest); câu “nghe hợp lý nhưng sai”.
- Có thể có snippet Java/Kotlin rất ngắn: mục tiêu là đọc hiểu API/từ khóa, không cần thuộc cú pháp rườm rà.

Checklist làm bài nhanh

- Nhìn từ khóa: Manifest? permission? lifecycle? adapter? service? notification? firebase? -> chuyển tới mục tương ứng.
- Gặp câu “khi nào nên dùng ...”: trả lời theo mục tiêu (đơn giản/linh hoạt/hiệu năng/compat).
- Gặp câu “callback nào chạy trước/sau”: dùng flow mũi tên + tình huống HOME/BACK/rotate.
- Luôn nhớ các “must-know bẫy”: Notification Channel API 26+, Service không tự tạo thread, savedInstanceState null/not-null.

💡 Chiến thuật xử lý đáp án "gần giống nhau":

Khi thấy 4 đáp án chỉ khác nhau 1-2 từ khóa, hãy kiểm tra theo thứ tự ưu tiên:

1. **Cấu trúc câu lệnh:** Kiểm tra xem có `.show()` (Toast), `.commit()` (Transaction/Editor), hay `.layoutManager` (RecyclerView) không. Thiếu những cái này là "vô hình".
2. **Đối tượng thực thi:** Phương thức đó thuộc về `Activity`, `Fragment`, hay `Context`? (Ví dụ: `this` trong Listener thường bị sai, phải là `MainActivity.this`).
3. **Luồng (Thread):** Câu lệnh đó có đang cố update UI từ `doInBackground` không? Nếu có -> Loại ngay.
4. **Giá trị mặc định:** Đề hay hỏi trạng thái lúc mới khởi tạo (thường là `null`, `false`, hoặc `-1`).

📋 Danh sách "Từ khóa nhạy cảm" (Nhìn thấy là phải soi kỹ):

- **"Always":** Trong Android ít khi có gì là "luôn luôn", thường phụ thuộc vào API Level.
- **"Automatically":** Kiểm tra xem hệ thống tự làm hay mình phải gọi lệnh (ví dụ: `RecyclerView` không tự hiển thị nếu thiếu `LayoutManager`).

- **"Immediately"**: Các tác vụ như `apply()` của `SharedPreferences` hay `commit()` của `Fragment` không phải lúc nào cũng thực thi tức thì một cách đồng bộ.

Bẫy hay gặp (đánh dấu đỏ khi học)

Danh sách bẫy:

Chủ đề	Ý chính
Notification Channel	Từ Android 8.0 (API 26) trở lên phải tạo channel trước khi notify.
Service & Thread	Service chạy nền nhưng code vẫn có thể chạy main thread; tác vụ nặng phải tự tạo thread/async.
savedInstanceState	null khi chạy lần đầu; not-null khi restore (rotate / hệ thống kill rồi khôi phục).
Permission	Khai báo trong Manifest ≠ đã được cấp (Android 6+ cần runtime request).
Adapter/ListView	ListView không tự có dữ liệu nếu không setAdapter; bind item chủ yếu ở getView().
RTDB vs Firestore	RTDB là JSON tree + realtime; Firestore là collection/document, query mạnh hơn (hay bị nhầm).
R class	<code>R.layout.main</code> hay <code>R.string.app_name</code> thực chất là số nguyên (int) , không phải là đối tượng View hay String.
Invisible vs Gone	INVISIBLE : Ẩn nhưng vẫn giữ chỗ . GONE : Ẩn và mất chỗ (View khác tràn vào).
Main Thread	Mặc định Android cấm gọi Network trên Main Thread (<code>NetworkOnMainThreadException</code>).
Implicit Intent	Nếu không có app nào nhận Intent ngầm định -> App sẽ bị Crash .

Context	<code>getApplicationContext()</code> dùng cho tác vụ lâu dài; <code>this</code> (Activity) dùng cho UI (như Dialog).
---------	---

Bảng quy đổi nhanh Kotlin ↔ Java (để đọc đề)

Tra nhanh:

Chủ đề	Kotlin	Java
Tạo Intent	<code>Intent(this, A::class.java)</code>	<code>new Intent(this, A.class)</code>
Put extra	<code>intent.putExtra("k", v)</code>	<code>intent.putExtra("k", v);</code>
Get string extra	<code>intent.getStringExtra("k")</code>	<code>getIntent().getStringExtra("k");</code>
Click listener	<code>btn.setOnClickListener { ... }</code>	<code>btn.setOnClickListener(new View.OnClickListener(){...});</code>
Toast	<code>Toast.makeText(ctx, "msg", Toast.LENGTH_SHORT).show()</code>	Giống Kotlin
Listener & Biến	Dùng biến thoải mái	Biến cục bộ truyền vào phải là <code>final</code>
Truy cập Activity	<code>this</code>	<code>MainActivity.this</code> (vì <code>this</code> đơn thuần là trỏ vào Listener)
Khai báo biến	<code>val</code> (read-only), <code>var</code> (mutable)	<code>final String</code> vs <code>String</code>
Null Safety	<code>object?</code> (nullable), <code>!!</code> (ép buộc)	Phải check <code>if (obj != null)</code>
Static	<code>companion object { ... }</code>	<code>static { ... }</code>

1) Android Core: Components – Manifest – Intent – Permissions

Tip làm bài trắc nghiệm: > Khi thấy đáp án kiểu: "Activity tự động lưu dữ liệu vào Database khi xoay màn hình", hãy cảnh giác. Activity chỉ tự lưu **UI State** vào **Bundle** thông qua **onSaveInstanceState**, còn Database là việc của lập trình viên phải tự làm.

[SYNTAX NHẬN DIỆN NHANH]

- Intent Tường minh (Explicit): `Intent(context, SecondActivity::class.java)` -> Chỉ đích danh.
- Intent Ngầm định (Implicit): `Intent(Intent.ACTION_DIAL, Uri.parse("tel:0123"))` -> Dựa vào Action.
- Gửi/Nhận dữ liệu: * Gửi: `intent.putExtra("ID", 123)`
 - Nhận: `val id = intent.getIntExtra("ID", 0)` (Luôn có giá trị mặc định là 0 nếu không tìm thấy).
- Khai báo Manifest (Bắt buộc):
`<activity android:name=".MainActivity" android:exported="true">`

`<intent-filter>`

`<action android:name="android.intent.action.MAIN" />`

`<category android:name="android.intent.category.LAUNCHER"`

`/>`

`</intent-filter>`

`</activity>`

`<uses-permission android:name="android.permission.CAMERA" />`

- Gửi dữ liệu: `intent.putExtra("key", value)`
- Nhận dữ liệu: `intent.getTypeExtra("key")` (ví dụ `getIntExtra`, `getStringExtra`).
- Runtime Permission (API 23+): * Check:
`ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)`
 - Request: `ActivityCompat.requestPermissions(this, arrayOf(Manifest.permission.CAMERA), REQ_CODE)`

- **Vòng đời Service:** `startService(intent)` -> `onStartCommand`
| `stopService(intent)` -> `onDestroy`.

1.1 Application Components (Building Blocks)

4 component chính:

Component	Dùng khi	Điểm nhớ nhanh
Activity	Màn hình/UI, tương tác người dùng	Có lifecycle; navigation qua Intent
Service	Tác vụ nền (nhạc, sync, upload...)	Không tự tạo thread; có thể start/bind
BroadcastReceiver	Nhận sự kiện hệ thống/app	Callback onReceive; normal vs ordered
ContentProvider	Chia sẻ data giữa app	Dùng URI; ít gặp nếu không học sâu

Thêm LocalBroadcastManager: Chỉ hoạt động bên trong app. Ưu điểm: Bảo mật hơn (app khác không chặn được tin nhắn) và hiệu năng cao hơn.

Service (Chi tiết "Sống còn"):

- **Nguyên tắc:** Service chạy trên **Main Thread** -> Tác vụ nặng (MP3, Network) phải tự tạo Thread riêng.
- **Lifecycle:** `onCreate()` -> `onStartCommand()` (hoặc `onStart()`) -> `onDestroy()`.
- **Điều khiển:**
 - `startService()`: Chạy vô thời hạn cho đến khi gọi `stopService()` hoặc `stopSelf()`.
 - `bindService()`: Chạy phụ thuộc vào component gọi nó (Client-Server).
- **Khai báo:** Bắt buộc phải có thẻ `<service>` trong Manifest.
- **Bẫy luồng:** Đề hay hỏi: "Service giúp tránh ANR (Application Not Responding) đúng hay sai?". **Sai**, vì nó chạy trên Main Thread. Muốn tránh ANR phải tự tạo Thread/Coroutine bên trong Service.
- **Bẫy onStartCommand return types:**
 - `START_STICKY`: Hệ thống kill -> tự tạo lại Service nhưng Intent gửi đến là `null`.
 - `START_NOT_STICKY`: Hệ thống kill -> không tự tạo lại trừ khi có Intent mới.

- **START_REDELIVER_INTENT**: Tự tạo lại và gửi lại Intent cuối cùng (dùng cho việc đang download dở).
- **Lifecycle đặc biệt**: `onCreate()` chỉ chạy **1 lần duy nhất** dù gọi `startService()` 10 lần. Nhưng `onStartCommand()` sẽ chạy **10 lần**.
- **Stop Service**: `stopSelf()` (tự sát) vs `stopService(Intent)` (bị giết từ ngoài). Cả hai đều dẫn tới `onDestroy()`.

Broadcast Receiver:

- **Hành vi**: `onReceive()` chạy trên Main Thread và có giới hạn thời gian (thường 10s). Không được làm việc nặng ở đây.
- **Static vs Dynamic**:
 - *Static (Manifest)*: App không chạy vẫn nhận được (trừ một số quyền từ Android 8+ bị hạn chế).
 - *Dynamic (Code)*: Phải `registerReceiver` và **bắt buộc** `unregisterReceiver` (thường ở `onStop` hoặc `onDestroy`) để tránh leak.

1.2 AndroidManifest.xml

Manifest khai báo gì?

- Khai báo components: `<activity>`, `<service>`, `<receiver>`, `<provider>`.
- Khai báo permission dùng: `<uses-permission>`.
- Khai báo intent-filter cho Activity nhận intent (đặc biệt MAIN/LAUNCHER).
- Khai báo min/target SDK và metadata cấu hình khác.

Ghi rõ định nghĩa **targetSdkVersion**: Là phiên bản API mà ứng dụng đã được thử nghiệm và đảm bảo hoạt động ổn định.

Cái gì bắt buộc:

- **Thuộc tính `android:exported`**: Nếu Activity có `<intent-filter>`, mặc định `exported` là `true` (app khác mở được). Nếu không có filter, mặc định là `false`. Từ Android 12, phải khai báo rõ `true/false` nếu có filter, nếu không sẽ lỗi cài đặt.
- **Bẫy thẻ**: `<uses-permission>` (xin quyền) nằm ngoài thẻ `<application>`. Các thẻ `<activity>`, `<service>`... phải nằm **trong** thẻ `<application>`.
- **Thứ tự ưu tiên**: Intent-filter nào có `priority` cao hơn sẽ được gọi trước (trong Ordered Broadcast).

Thẻ hay gặp:

Thẻ	Ý nghĩa nhanh
-----	---------------

<activity>	Khai báo màn hình; có thể kèm intent-filter
<intent-filter>	Định nghĩa Activity nhận intent nào (action/category/data)
<uses-permission>	Yêu cầu quyền (vd: INTERNET, CAMERA, ACCESS_FINE_LOCATION)
<service>	Khai báo service để hệ thống biết
<receiver>	Khai báo receiver (static) để nhận broadcast

1.3 Intent & Navigation

Explicit vs Implicit intent:

Loại	Dấu hiệu	Ví dụ
Explicit	Chỉ rõ component đích	Mở Activity A trong app của mình
Implicit	Không chỉ rõ component; dựa trên intent-filter	Mở trình duyệt/Share/Call...

- **Task/Back stack keyword:** `FLAG_ACTIVITY_NEW_TASK`, `CLEAR_TOP`, `SINGLE_TOP` (thầy hay hỏi “bấm back về đâu?”).
- **Implicit intent fail:** phải `resolveActivity(packageManager)` trước khi `startActivity` (tránh crash).
- **Bundle vs Intent extras:** khi nào dùng Bundle, khi nào dùng extra.

Intent (So sánh & PendingIntent):

- **Explicit (Trường minh):** `Intent(context, TargetActivity::class.java)`. An toàn nhất vì chỉ đích danh.
- **Implicit (Ngầm định):** Dựa vào 3 yếu tố để filter: `Action` (làm gì), `Category` (ngữ cảnh), `Data` (kiểu dữ liệu - MIME type).

- **Bẫy:** Nếu có nhiều app cùng nhận được 1 Implicit Intent -> Hệ thống hiện App Chooser. Nếu không có app nào nhận -> **App Crash**.
- **PendingIntent (Ủy quyền):** * **Khái niệm:** Một "bao nilon" bọc Intent lại và đưa cho hệ thống (Notification Manager, Alarm Manager).
 - **Behavior:** Hệ thống sẽ thực thi Intent đó với danh nghĩa (quyền) của App bạn, ngay cả khi App bạn đã bị kill.

MAIN/LAUNCHER

- Activity có action MAIN + category LAUNCHER sẽ xuất hiện icon để mở app từ launcher.

Activity được quản lý như thế nào trong Stack (Task):

- **Standard (Mặc định):** Luôn tạo instance mới (mở 10 lần là có 10 Activity trong stack).
- **SingleTop:** Nếu Activity đó đang ở **ĐỈNH** stack thì không tạo mới, chỉ gọi `onNewIntent()`.
- **SingleTask:** Tạo một task mới, Activity đó là duy nhất trong task đó.

1.4 Permissions (runtime)

Flow runtime permission (Android 6+)

- 1) Kiểm tra: đã được cấp chưa?
- 2) Nếu chưa: `requestPermissions(...)`
- 3) Nhận kết quả: `onRequestPermissionsResult(...)`
- 4) Nếu user từ chối: cần nhắc hiển thị rationale/điều hướng Settings (tùy yêu cầu).

(Security):

- **Keystore System:** Dùng để lưu trữ khóa mã hóa và dữ liệu nhạy cảm một cách an toàn nhất.
- **TLS/SSL:** Mã hóa dữ liệu trên đường truyền mạng (HTTPS).

Permissions (Ranh giới API):

- Phân loại:
 - **Normal:** (Internet, Bluetooth...) Tự cấp khi cài đặt, chỉ cần khai báo Manifest.
 - **Dangerous:** (Camera, Location, Contacts...) Bắt buộc khai báo Manifest **VÀ** xin quyền lúc Runtime (Android 6.0+ / API 23+).

- **Flow chuẩn:** `checkSelfPermission` -> (Nếu chưa có) `requestPermissions` -> `onRequestPermissionsResult`.
 - **Bẫy:** Nếu user chọn "Don't ask again", lần sau gọi `requestPermissions` sẽ không hiện popup nữa mà trả về Denied ngay lập tức.

1.5 Android Stack & Architecture

- **Linux Kernel:** Quản lý driver, bộ nhớ, năng lượng.
- **HAL (Hardware Abstraction Layer):** Cầu nối giữa thư viện Java và phần cứng.
- **Android Runtime (ART) & Native C/C++ Libraries:** ART dùng **AOT (Ahead-of-Time)** biên dịch app khi cài đặt để chạy nhanh hơn.
- **Java API Framework:** Các Manager (Activity Manager, Resource Manager...).
- **System Apps:** Các ứng dụng hệ thống (Contact, Phone...).

ADB (Android Debug Bridge): Công cụ dòng lệnh (command-line tool) cho phép máy tính giao tiếp với thiết bị Android (cài đặt app, sao chép file, xem log).

Thêm **ProGuard/R8:** Công cụ tối ưu hóa, rút gọn dung lượng và làm rối mã nguồn (Obfuscation) để bảo mật.

Thêm **Architecture Components:** Bao gồm ViewModel, LiveData, Room, Paging, và WorkManager. (Ghi chú: RecyclerView KHÔNG thuộc nhóm này).

Android Stack & Architecture (Tiểu tiết ART vs Dalvik):

- **Dalvik (Cũ):** Dùng **JIT (Just-In-Time)** - biên dịch lúc đang chạy app -> Tốn CPU, tiết kiệm bộ nhớ cài đặt.
- **ART (Mới - từ Android 5.0):** Dùng **AOT (Ahead-of-Time)** - biên dịch lúc cài đặt -> App chạy mượt hơn, cài lâu hơn, tốn bộ nhớ hơn.
- **HAL (Hardware Abstraction Layer):** Mục đích là để các nhà sản xuất phần cứng không cần công khai mã nguồn driver mà vẫn kết nối được với Android framework.

1.6 Launch Modes & Context

- `android:launchMode` (Trong Manifest):
 - **standard** (Mặc định): Mỗi lần start là tạo một instance mới.
 - **singleTop:** Nếu Activity đã ở **đỉnh** stack thì không tạo mới, gọi `onNewIntent()`.
 - **singleTask:** Tạo một Task mới, Activity đó là root (duy nhất trong task).
- Các loại Context:
 - **Application Context:** Tồn tại suốt đời app, dùng cho Database, Network, Library. **Bẫy:** Không dùng để hiện Dialog/Toast (trừ một số trường hợp đặc biệt).

- **Activity Context:** Dùng cho UI, Dialog, hiển thị giao diện.

2) Lifecycle & State: Activity/Fragment – onSaveInstanceState

- Khởi động Activity:
 - `startActivity(intent)` -> Mở bình thường.
 - `startActivityForResult(intent, REQ_CODE)` -> Mở để lấy kết quả (Kiểu cũ).
 - `registerForActivityResult(...)` -> Cách mở lấy kết quả hiện đại (Kiểu mới).
- Lưu trạng thái (Bundle):

```
override fun onSaveInstanceState(outState: Bundle) {  
    outState.putString("key", "giá trị")  
    super.onSaveInstanceState(outState)  
}
```
- Khôi phục trạng thái (Trong onCreate):

```
val data = savedInstanceState?.getString("key") // Dấu ? vì lần đầu chạy sẽ null
```
- Fragment Transaction (Bắt buộc phải có `commit()`):

```
supportFragmentManager.beginTransaction().replace(R.id.container,  
MyFragment()) // Thay thế fragment cũ .addToBackStack(null) // Quan trọng: Cho phép nhấn BACK để quay lại fragment trước .commit()
```
- **Inflate UI Fragment:** `return inflater.inflate(R.layout.fragment_x, container, false)`

Checklist "Bắt" cho Mục 2 (Nhớ kỹ trước khi khoan):

- **onRestart():** Luôn chạy flow: `onStop()` → `onRestart()` → `onStart()` → `onResume()`. (Không bao giờ chạy lại `onCreate()`).
- **onRestoreInstanceState():** Chạy sau `onStart()`. Bundle ở đây luôn `not-null`.
- **Finish():** Khi gọi lệnh `finish()`, Activity đi thẳng từ `onPause` → `onStop` → `onDestroy`.
- **Fragment Lifecycle:** Fragment chỉ tương tác được với UI sau khi `onActivityCreated()` (hoặc `onViewCreated()`) chạy xong.
- **Activity onPause()** thì **Fragment cũng onPause()** (Fragment luôn đi theo Activity mẹ).

2.1 Activity Lifecycle (flow)

Flow chuẩn

- onCreate → onStart → onResume → (running) → onPause → onStop → onDestroy
- onRestart xảy ra khi Activity từ stopped quay lại.

Xoay màn hình không phải là thứ duy nhất "giết" Activity.

- Nguyên nhân gây Destroy/Create lại Activity:
 - Thay đổi ngôn ngữ hệ thống (Localization).
 - Thay đổi kích thước font chữ (Accessibility).
 - Cắm/Rút bàn phím vật lý.
 - Thay đổi chế độ Dark Mode/Light Mode.
- Bẫy: Nếu khai báo `android:configChanges="orientation|screenSize"` trong Manifest, hệ thống sẽ KHÔNG hủy Activity nữa mà gọi hàm `onConfigurationChanged()`.

“**Câu lệnh khởi động Activity mới**” → cần ghi rõ: `startActivity(intent)` (và nếu có: `startActivityForResult` kiểu cũ / `ActivityResultLauncher` kiểu mới).

Ranh giới giữa **Pause** và **Stop**:

Trạng thái	Callback cuối cùng	Đặc điểm hiển thị	Khả năng tương tác
Resumed	<code>onResume()</code>	Hiển thị đầy đủ	Có (Has Focus)
Paused	<code>onPause()</code>	Vẫn thấy một phần (ví dụ: bị một Dialog trong suốt hoặc nửa màn hình che)	Không (Lost Focus)
Stopped	<code>onStop()</code>	Bị che khuất hoàn toàn (ví dụ: bấm HOME, mở Activity khác full màn hình)	Không

Tình huống hay hỏi:

Tình huống	Chuỗi callback thường gặp
Bấm HOME	<code>onPause</code> → <code>onStop</code> (vẫn còn trong bộ nhớ)

Bấm BACK để thoát	onPause → onStop → onDestroy
Mở Activity khác phủ lên	onPause (→ có thể onStop nếu bị che hoàn toàn)
Rotate / config change	onPause → onStop → onDestroy → onCreate → onStart → onResume
Hệ thống kill rồi restore	onCreate nhận savedInstanceState != null; state được restore

- **Back Stack:** Activity được quản lý theo cơ chế **LIFO** (Last In, First Out). **Fragment**
- **Lifecycle (Trạng thái đặc biệt):**
 - o **onAttach():** Khi Fragment được gắn vào Activity.
 - o **onCreateView():** Nơi tạo giao diện cho Fragment.
 - o **onDestroyView():** Khi giao diện của Fragment bị gỡ bỏ (nhưng Fragment vẫn tồn tại).

Các tình huống bẫy "tiểu tiết":

- **Tình huống Dialog:** Khi một Dialog xuất hiện (dạng **AlertDialog**), Activity phía dưới **KHÔNG** rơi vào **onPause()** (vì Dialog không phải là Activity). Tuy nhiên, nếu là một **Activity** được định dạng kiểu Dialog (**Theme.Dialog**), thì Activity dưới sẽ dính **onPause()**.
- **onRestart():** Luôn đi kèm với **onStart()**. Thứ tự: **onStop()** -> **onRestart()** -> **onStart()** -> **onResume()**. Lưu ý: **onCreate()** **không** chạy lại trong flow này.
- **Xoay màn hình (Configuration Change):** Đây là quá trình "Huỷ để xây lại". Activity cũ bị **onDestroy()**, Activity mới được tạo với **onCreate()** kèm theo **Bundle** dữ liệu cũ.

2.2 onSaveInstanceState vs onPause/onResume

Mẹo trắc nghiệm: Nếu đề hỏi "Callback nào chắc chắn được gọi trước khi Activity bị hệ thống kill?", đáp án thường là **onPause()**.

onSaveInstanceState không được gọi nếu user tự bấm BACK để thoát app (vì user chủ động thoát thì không cần save).

Vì sao cần onSaveInstanceState?

- onPause/onResume phục vụ chuyển trạng thái tạm thời; không đảm bảo dữ liệu UI sẽ được khôi phục sau khi hệ thống kill process.
- onSaveInstanceState lưu "UI state nhỏ" để hệ thống khôi phục sau rotate/process death.
- Dữ liệu lớn: lưu DB/File/Prefs, không nhét vào Bundle.
- Tham số **bundle** trong **onCreate**:
 - **null**: Khi Activity được khởi tạo lần đầu tiên.
 - **not-null**: Khi Activity được tái tạo sau khi bị hệ thống hủy (do xoay màn hình, thiếu bộ nhớ).
- **Tại sao cần onSaveInstanceState?** Vì **onPause** chỉ dùng để tạm dừng tương tác, còn **onSaveInstanceState** mới thực sự lưu lại "trạng thái đóng băng" của UI vào Bundle để khôi phục khi cần.

savedInstanceState trong onCreate:

Giá trị	Khi nào
null	Chạy lần đầu hoặc không có state để restore
not-null	Sau rotate/config change hoặc hệ thống kill rồi restore

Sự khác biệt giữa việc lưu trong **onPause** và **onSaveInstanceState**:

- **Ranh giới thời gian:** **onSaveInstanceState()** thường được gọi **sau** **onPause()** và **trước** **onStop()**.
- **Bẫy Bundle:**
 - Dữ liệu trong **onSaveInstanceState(outState: Bundle)** chỉ tồn tại trong bộ nhớ RAM tạm thời. Nếu user chủ động tắt app (vuốt bỏ ở Recent Apps), Bundle này sẽ bị xóa sạch.
 - **Tham số trong onCreate(savedInstanceState: Bundle?):** Nếu app chạy lần đầu, nó là **null**. Nếu app bị hệ thống kill hoặc xoay màn hình, nó mới **not-null**.

2.3 Fragment essentials

- “XML nào định nghĩa UI của Fragment?” → cần ghi rõ: file nằm trong `res/layout/fragment_*.xml`, inflate trong `onCreateView()`.
- “Fragment giao tiếp với Activity bằng cách nào?” → chốt 2–3 cách “đúng sách”: Interface callback, Shared ViewModel (nếu học), hoặc `FragmentManager#setFragmentResult`.
- **Fragment UI file:** `res/layout/fragment_x.xml`, inflate:
`inflater.inflate(R.layout.fragment_x, container, false)`
- **Gắn Fragment:** `FragmentManager#beginTransaction().commit()`
- **BackStack:** có `addToBackStack()` thì BACK quay lại fragment, không có thì thoát Activity.
- Tiểu tiết thời điểm
 - `commit()`: Là lệnh chạy **bất đồng bộ** (asynchronous). Nó xếp hàng chờ hệ thống rảnh mới chạy, không thực hiện ngay lập tức.
 - `executePendingTransactions()`: Nếu muốn lệnh `commit()` ở trên phải chạy **ngay lập tức** thì gọi thêm hàm này.

Checklist "Bẫy" cho Mục 2:

1. **Nút BACK vs Nút HOME:**
 - BACK = Huỷ (`onDestroy`).
 - HOME = Tạm dừng và Dừng (`onPause` -> `onStop`), không huỷ.
2. **`onRestoreInstanceState()`:** Callback này được gọi **sau** `onStart()`.
Bạn có thể lấy lại dữ liệu ở `onCreate` hoặc `onRestoreInstanceState`, nhưng ở `onCreate` bạn phải check `null`, còn ở `onRestoreInstanceState` thì Bundle luôn `not-null`.
3. **`Finish()`:** Khi gọi lệnh `finish()`, Activity sẽ đi thẳng một mạch từ `onPause` -> `onStop` -> `onDestroy`.

Fragment là gì?

- Một phần UI trong Activity; 1 Activity có thể chứa nhiều Fragment.
- Thường dùng cho UI phức tạp, tablet/multi-pane, hoặc navigation theo fragment.

Fragment có vòng đời phức tạp hơn vì nó phụ thuộc vào Activity "mẹ".

- Cặp bài trùng Fragment - Activity:
 - Activity `onPause()` -> Fragment `onPause()`.
 - Fragment chỉ có thể tương tác với UI sau khi `onActivityCreated()` (hoặc `onViewCreated()`) chạy xong.
- `FragmentManager` (Bẫy replace vs add):
 - **`add()`:** Chồng thêm một Fragment lên trên. Fragment cũ vẫn "Sống" (không bị Destroy view).

- **replace():** = `remove()` Fragment cũ + `add()` Fragment mới. Fragment cũ sẽ bị `onDestroyView()`.
- **addToBackStack():** Nếu không gọi lệnh này, khi bấm nút BACK, app sẽ thoát luôn Activity thay vì quay lại Fragment trước đó.

Fragment lifecycle (rút gọn):

Nhóm	Callback
Attach/Create	onAttach → onCreate
UI	onCreateView → onCreateView → onStart → onResume
Pause/Stop	onPause → onStop
Destroy	onDestroyView → onDestroy → onDetach

FragmentManager keywords

- add / replace / remove / commit (và addToBackStack nếu muốn BACK quay lại fragment trước).

3) UI: Layouts – Adapter/ListView – WebView – Toast/Dialog

[SYNTAX NHẬN DIỆN NHANH]

- ConstraintLayout (XML):
 - `android:layout_width="0dp" -> MATCH_CONSTRAINT` (Giãn theo ràng buộc).
 - `app:layout_constraintHorizontal_bias="0.3" -> Lệnh trái 30%.`
 - `app:layout_constraintLeft_toLeftOf="parent" -> Ràng buộc cạnh.`
- RecyclerView (Adapter):
 - `onCreateViewHolder: val view = LayoutInflater.from(parent.context).inflate(R.layout.item, parent, false)`
 - `onBindViewHolder: holder.textView.text = data[position]`
 - `LayoutManager: recyclerView.layoutManager = LinearLayoutManager(this)`
- WebView:
 - `webView.settings.javaScriptEnabled = true` (Bật Script).
 - `webView.loadUrl("https://google.com")`
- Pop-ups & Messages:
 - `Toast.makeText(this, "Msg", Toast.LENGTH_SHORT).show()` (Bắt buộc có `.show()`).
 - `Snackbar.make(view, "Msg", Snackbar.LENGTH_LONG).setAction("Undo") { ... }.show()`
 - `AlertDialog.Builder(this).setTitle("Hi").setPositiveButton("OK") { ... }.create().show()`
- View Update:
 - `invalidate()` -> Vẽ lại nội dung (`onDraw`).
 - `requestLayout()` -> Tính lại kích thước (`onMeasure`).

Checklist "Bẫy" cho Mục 3:

- Invisible vs Gone:
 - **INVISIBLE**: Ẩn nhưng vẫn **giữ chỗ** trên màn hình.

- **GONE**: Ẩn hoàn toàn, các View khác sẽ tràn vào **chiếm chỗ**.
- **RecyclerView Component**: Đáp án đúng phải có đủ 3 cái: **Adapter**, **ViewHolder**, **LayoutManager**. Thiếu 1 trong 3 là sai.
- **Context**: Trong Activity dùng **this**. Trong Fragment phải dùng **requireContext()** hoặc **getActivity()**. Nếu đề cho **getApplicationContext()** thì thường dùng cho Toast, nhưng không dùng cho Dialog (Dialog cần Activity Context).
- Activity context: **this**
- Fragment: **requireContext()** / **requireActivity()**
- Application context: **applicationContext** (dùng cho thứ sống lâu; tránh cho Dialog)
- **JavaScript**: Đề hỏi "Tại sao WebView không chạy được Script?", đáp án là do **javaScriptEnabled** mặc định là **false**.
- **"Dùng px để đảm bảo độ sắc nét trên mọi màn hình"**: Sai. Dùng px làm UI bị vỡ layout trên màn hình mật độ cao.
- **"sp và dp luôn luôn bằng nhau về kích thước vật lý"**: Sai. Chúng chỉ bằng nhau khi người dùng để cài đặt font hệ thống ở mức mặc định (Normal).
- **"1 dp luôn bằng 1/160 inch"**: Đúng (theo lý thuyết chuẩn của Android để đảm bảo tính nhất quán vật lý).
- **Mối quan hệ toán học (Công thức tính)**: Thầy có thể cho snippet hoặc hỏi công thức đổi từ dp sang px: $px = dp * (dpi / 160)$
- Màn hình chuẩn (**Baseline**) là **160 dpi** (gọi là mức **mdpi**). Tại đây: 1 dp = 1 px.
- **Bắt sp**: Nếu đề hỏi "Đơn vị nào giúp hỗ trợ người khiếm thị/người già (Accessibility)?", câu trả lời luôn là **sp**. Nếu dùng dp cho chữ, khi người dùng chỉnh font hệ thống to lên, chữ trong app vẫn sẽ nhỏ -> Sai nguyên tắc thiết kế.
- **Bắt dpi**: dpi không phải là đơn vị đo kích thước. Nếu đáp án liệt kê dpi cùng nhóm với dp, sp để đo chiều cao View là Sai. dpi là thông số của màn hình để hệ thống chọn ảnh từ thư mục **res/drawable-xxxx**.
- **Phân cấp mật độ (DPI Levels)**:
 - **mdpi**: 160 dpi (tỉ lệ 1x)
 - **hdpi**: 240 dpi (tỉ lệ 1.5x)
 - **xhdpi**: 320 dpi (tỉ lệ 2x)
 - **xxhdpi**: 480 dpi (tỉ lệ 3x)

3.1 MVC (mức khái niệm)

MVC trong Android (ý nghĩa trắc nghiệm)

- Model: dữ liệu + logic nghiệp vụ
- View: UI hiển thị
- Controller: xử lý sự kiện, điều phối (Activity/Fragment thường đóng vai trò controller)

3.2 Layout selection (Linear/Relative/Constraint)

Chọn layout nhanh:

Layout	Nên dùng khi	Tránh khi
LinearLayout	UI đơn giản theo 1 chiều (dọc/ngang)	Nhiều level lồng nhau
RelativeLayout	Căn canh theo view khác (cũ, ít dùng hơn)	UI phức tạp/responsive hiện đại
ConstraintLayout	UI phức tạp, responsive, giảm nested layout	Khi bạn chỉ cần linear đơn giản

Nhấn mạnh **LinearLayout**: Layout tối ưu nhất để xếp View theo một hàng hoặc một cột (dùng thuộc tính **orientation**).

ConstraintLayout – keyword

- Constraint = ràng buộc cạnh với parent hoặc view khác; cần đủ constraint để tránh ambiguous.
- ConstraintSet: clone → connect → applyTo (nếu làm programmatically).
- ConstraintLayout (Vua của các loại Layout):
 - **0dp (MATCH_CONSTRAINT)**: Trong ConstraintLayout, **không dùng match_parent**. Để một View giãn ra hết các ràng buộc, ta dùng `android:layout_width="0dp"`.
 - **Bias**: `layout_constraintHorizontal_bias="0.3"` (đẩy View về bên trái 30% khoảng trống).

- **Chains:** Cần ít nhất 2 View ràng buộc lẫn nhau. Thuộc tính `layout_constraintHorizontal_chainStyle` có 3 giá trị: `spread` (mặc định), `spread_inside`, `packed`.
- **LinearLayout:** Thuộc tính quan trọng nhất là `android:orientation` (ngang/dọc) và `android:weightSum` kết hợp với `android:layout_weight` để chia tỉ lệ màn hình.

3.3 Views & event handling

Nhớ nhanh

- `setOnClickListener / setOnLongClickListener`
- `setVisibility: VISIBLE / INVISIBLE / GONE`
- `requestFocus, enabled/disabled`
- `setContentView(...)`
- `findViewById(...)`
- `LayoutInflater.inflate(...)`

View Binding: Thay thế `findViewById`, an toàn về kiểu dữ liệu (type-safe) và an toàn về null (null-safety).

Thêm **AnimatorSet:** Công cụ để nhóm và điều phối nhiều Animation chạy đồng thời (together) hoặc tuần tự (sequentially).

Thứ tự vẽ giao diện:

- **Thứ tự hàm chạy:** `onMeasure()` (Tính kích thước) -> `onLayout()` (Xác định vị trí) -> `onDraw()` (Vẽ lên Canvas).
- **Hành vi update:**
 - `invalidate()`: Chỉ bắt hệ thống chạy lại `onDraw()` (vẽ lại nội dung).
 - `requestLayout()`: Bắt hệ thống chạy lại từ đầu `onMeasure()` (tính lại kích thước).

3.4 ListView & Adapter & RecyclerView

- **Tạo View:** `onCreateViewHolder` \$ \rightarrow \$ dùng `LayoutInflater.inflate(...)`.
- **Đổ Data:** `onBindViewHolder` \$ \rightarrow \$ `holder.textView.text = data[position]`.
- **ConstraintLayout Bias:** `app:layout_constraintHorizontal_bias="0.3"` (Số 0.3 nghĩa là lệch trái 30%).
- **“Sự kiện khi chọn item ListView là gì?”** → thiếu keyword: `setOnClickListener` (và `onItemClick(...)`).
- ListView click: `setOnClickListener`

- RecyclerView bắt click item: thường **tự viết** (listener trong ViewHolder)
→ bẫy “RecyclerView có sẵn OnItemClickListener không?” (không có built-in như ListView)
- Bẫy phương thức:
 - **ListView**: Dùng `getView()`. Mọi thứ xử lý trong 1 hàm duy nhất.
 - **RecyclerView**: Chia làm 2 bước rõ rệt:
 1. `onCreateViewHolder()`: **Thổi phồng (Inflate)** XML layout cho item và tạo ViewHolder. (Chỉ chạy khi cần tạo mới).
 2. `onBindViewHolder()`: **Gán dữ liệu** từ mảng vào các View đã có. (Chạy liên tục khi cuộn).
- **LayoutManager**: Nếu thiếu dòng `recyclerView.setLayoutManager = ...`, app vẫn chạy nhưng màn hình **trắng tinh** (không hiện lỗi crash nhưng không hiện dữ liệu).
- **Cập nhật dữ liệu**: Luôn dùng `adapter.notifyDataSetChanged()` để báo cho View vẽ lại khi nguồn dữ liệu (List/ArrayList) thay đổi.

Workflow ListView

- 1) Tạo item layout (XML)
- 2) Chuẩn bị data source
- 3) Tạo adapter (ArrayAdapter/BaseAdapter/custom)
- 4) `listView.setAdapter(adapter)`
- 5) Bind dữ liệu chủ yếu trong `getView()`

Adapter hay hỏi:

Adapter	Khi dùng
ArrayAdapter	Danh sách đơn giản (String, 1 layout)
BaseAdapter / Custom Adapter	Item phức tạp (nhiều field, custom view)

Card — RecyclerView

Định nghĩa: Widget hiển thị danh sách lớn, tối ưu bằng cơ chế tái sử dụng view (recycle).

3 thành phần chính để hoạt động (đáp án thường gặp):

1. **RecyclerView** (View)
2. **LayoutManager** (LinearLayoutManager/GridLayoutManager/...)
3. **Adapter** (kèm **ViewHolder**)

(nếu đề tính “đủ 3”, thường lấy: Adapter + LayoutManager + ViewHolder, nhưng cách an toàn nhất khi chọn đáp án là: Adapter + LayoutManager + ViewHolder/RecyclerView tùy câu)

Ai tái sử dụng View?

- **ViewHolder** + cơ chế recycle của RecyclerView (ViewHolder giữ reference view, được reuse).
- (Trong code: `onCreateViewHolder` tạo mới khi cần; `onBindViewHolder` gán dữ liệu khi reuse)

Phương thức gán dữ liệu vào ViewHolder:

- `onBindViewHolder(holder, position)`

Bấy hay hỏi:

- ListView dùng `getView()`; RecyclerView dùng `onCreateViewHolder` + `onBindViewHolder`
- RecyclerView **bắt buộc có LayoutManager** mới hiển thị được.

Các widget danh sách khác:

- **Spinner:** Chọn một giá trị từ tập hợp (dạng dropdown).
- **AutoCompleteTextView:** Gợi ý khi người dùng nhập văn bản.
- **HorizontalScrollView:** Cho phép cuộn ngang để chọn item (thường là ảnh hoặc màu sắc).
- **CardView:** Cần thư viện `androidx.cardview`. Thuộc tính quan trọng: `cardCornerRadius` (bo góc) và `cardElevation` (đổ bóng).

3.5 WebView

(Bấy cấu hình & Bảo mật)

- **Bắt buộc trong Manifest:** WebView sẽ không tải được trang web nếu thiếu `<uses-permission android:name="android.permission.INTERNET" />`.
- **loadUrl vs loadData:**
 - `loadUrl("https://...")`: Tải từ link.

- **loadData(htmlString, "text/html", "UTF-8")**: Tải trực tiếp đoạn code HTML có sẵn.
- **Hành vi nút Back**: Mặc định nút Back của điện thoại sẽ thoát App. Để nút Back quay lại trang web trước đó, phải ghi đè `onBackPressed()` và gọi `webView.goBack()`.

WebView dùng khi

- Hiển thị trang web trong app, nội dung HTML, hoặc một phần UI dựa trên web.
- Thường đi kèm `loadUrl/loadData` và cấu hình cơ bản (tùy yêu cầu môn).

Chi tiết quan trọng:

- **3 Chiến lược thiết kế**: Pure Android (Native), Pure External HTML (Browser), Mixed Mode (Local HTML trong app).
- **Phương thức điều hướng**: `canGoBack()` / `goBack()`, `canGoForward()` / `goForward()`, `reload()`.
- **Bắt JavaScript**: Mặc định JavaScript bị tắt. Phải bật bằng `webView.settings.javaScriptEnabled = true`.
- **Bắt Bảo mật**: Để gọi code Java/Kotlin từ JavaScript, phải dùng annotation `@JavascriptInterface` cho phương thức trong Interface.

3.6 - 3.10 Pop-up & Messages (Phân biệt tinh vi)

- Toast:
 - Phương thức: `Toast.makeText(context, text, duration).show()`. **Bẫy**: Quên `.show()` là lỗi phổ biến nhất.
 - Duration: Chỉ có 2 hằng số `LENGTH_SHORT` (~2s) và `LENGTH_LONG` (~3.5s). Truyền số khác vào là sai.
- Snackbar:
 - Cần một **View** (thường là layout cha) làm tham số đầu tiên thay vì `Context`.
 - Action: `snackbar.setAction("Undo") { ... }`.
- AlertDialog:
 - Dùng class `AlertDialog.Builder` để khởi tạo.
 - 3 loại Button: `setPositiveButton`, `setNegativeButton`, `setNeutralButton`.
 - **Hành vi**: `setCancelable(false)` khiến người dùng bắt buộc phải bấm nút, không thể bấm ra ngoài để tắt.
- Pickers (Bộ chọn):

- `DatePickerDialog` và `TimePickerDialog` đều có các phương thức listener để nhận kết quả khi user chọn xong.
- Thường dùng `Calendar.getInstance()` để lấy thời gian hiện tại làm mặc định cho Picker.

3.6 Toast & Dialog

Toast vs Dialog:

Tiêu chí	Toast	Dialog
Tương tác	Không tương tác (tự tắt)	Có tương tác (OK/Cancel...)
Mục tiêu	Thông báo nhanh	Yêu cầu xác nhận/nhập liệu
Thời gian	SHORT/LONG	Tồn tại đến khi user đóng

- **Đơn vị đo:**
 - **dp (density-independent pixels):** Dùng cho kích thước layout/view để đảm bảo hiển thị giống nhau trên các màn hình có độ phân giải khác nhau.
 - **sp (scale-independent pixels):** Dùng cho **Font chữ** (có thể thay đổi theo cài đặt của người dùng).
- **ConstraintLayout:**
 - **Guideline:** Đường kẻ phụ (ngang/dọc) không hiển thị trên UI, dùng để canh lề.
 - **Barrier:** Giống Guideline nhưng vị trí thay đổi dựa trên kích thước của các view nó bao quanh.
 - **Chains:** Kết nối một nhóm view theo chiều ngang hoặc dọc với khả năng phân bổ không gian (Spread, Inside, Weighted).

3.7 Toast vs Snackbar vs Dialog vs BottomSheet

Tiêu chí	Toast	Snackbar	Dialog	BottomSheet
Vị trí	Bất kỳ đâu (mặc định dưới)	Chỉ ở đáy màn hình	Giữa màn hình (modal)	Từ dưới lên (Bottom-up)
Tương tác	Không (not clickable)	Có (Action Button)	Có (Button/Input)	Có (Menu, List, View)
Đóng	Tự mất (timeout)	Tự mất hoặc Vuốt để đóng	User phải tương tác	Vuốt xuống/Bấm

				ngoài
Mục đích	Feedback nhanh	Feedback + Phục hồi (Undo)	Xác nhận quan trọng	Mở rộng tùy chọn/Menu

BẤY Snackbar: Gắn vào `CoordinatorLayout` sẽ có thêm tính năng (vuốt đóng, đẩy `FloatingActionButton` lên).

3.8 Pickers: DatePicker, TimePicker, NumberPicker (đảm bảo valid format).

`DatePicker`, `TimePicker` giúp đảm bảo dữ liệu ngày/giờ hợp lệ theo locale.

3.9 Custom UI: Các bước tạo Custom Toast và Custom Dialog (dùng `setView`).

Custom UI: Cả Toast và Dialog đều hỗ trợ `setView(layout)` để dùng XML tùy chỉnh.

3.10 BottomSheet – Khái niệm & Phân loại

- **Khái niệm:** Là một bề mặt (surface) hiển thị từ cạnh dưới màn hình lên trên, dùng để cung cấp thêm không gian cho các nội dung bổ sung hoặc danh sách hành động mà không làm mất ngữ cảnh hiện tại.
- **Phân loại trải nghiệm hay hỏi:**
 - **Persistent Bottom Sheet:** Hiển thị như một phần của giao diện (cùng cấp với các View khác), thường dùng để hiển thị thông tin phụ (ví dụ: trình phát nhạc thu nhỏ).
 - **Modal Bottom Sheet:** Hiển thị đè lên trên giao diện chính (như một Dialog), làm mờ màn hình phía sau. User phải đóng hoặc chọn một mục mới tiếp tục dùng app.
- **Thành phần quan trọng:**
 - **BottomSheetDialog:** Dùng khi muốn hiện UI đơn giản từ dưới lên.
 - **BottomSheetDialogFragment:** Dùng khi UI phức tạp, có vòng đời (lifecycle) riêng, dễ quản lý hơn.
 - **Behavior (Hành vi):** Có thể kéo (drag) để mở rộng hoặc thu hẹp (thường dùng `CoordinatorLayout` để quản lý).

3.11 Đơn vị đo lường:

Đơn vị	Tên đầy đủ	Đặc điểm hành vi (Behavior)	Trường hợp sử dụng
px	Pixel	Điểm ảnh vật lý. Cố định ,	Tránh dùng (khiến

		không co giãn theo mật độ màn hình.	UI bị bé tí trên màn hình độ phân giải cao).
dp / dip	Density-Independent Pixel	Pixel ảo. Tự động điều chỉnh dựa trên mật độ màn hình (dpi).	Kích thước View, Layout, Khoảng cách (Margin/Padding).
sp	Scale-Independent Pixel	Giống dp nhưng thêm hệ số tỉ lệ font chữ của người dùng.	Kích thước chữ (Text size) duy nhất.
dpi	Dots Per Inch	Mật độ điểm ảnh (số lượng pixel trên 1 inch).	Phân loại thiết bị (mdpi, hdpi, xhdpi...).
pt	Point	1/72 inch vật lý.	Ít dùng (thường thấy trong iOS hoặc in ấn).
in / mm	Inch / Millimeter	Kích thước vật lý chuẩn xác.	Rất ít dùng.

3.12 View Lifecycle & View Hierarchy

Thứ tự vẽ giao diện:

- View Lifecycle (Thứ tự hàm chạy):
 1. `onMeasure()`: Tính toán kích thước View.
 2. `onLayout()`: Xác định vị trí View.
 3. `onDraw()`: Vẽ nội dung lên màn hình (Canvas).
 - **Bắt:** Gọi `invalidate()` sẽ bắt hệ thống chạy lại `onDraw()`. Gọi `requestLayout()` sẽ chạy lại từ đầu (`onMeasure`).
- **View Hierarchy:** Cấu trúc dạng Cây (Tree). `ViewGroup` chứa các `View` con.

4) Resources & Local Storage: strings/fonts/preferences/files

[SYNTAX NHẬN DIỆN NHANH]

- Truy xuất Resource:
 - Lấy chuỗi: `val s = getString(R.string.hello)` (Lưu ý: `R.string.hello` chỉ là số `int`).
 - Mở file **Raw**:
`resources.openRawResource(R.raw.music_file)`
 - Mở file **Assets**: `assets.open("folder/data.txt")` (Dùng đường dẫn String).
- SharedPreferences (Lưu trữ Key-Value):
`val pref = getSharedPreferences("MyFile", Context.MODE_PRIVATE)`
// Ghi dữ liệu (Editor)
`pref.edit().putInt("score", 100).apply()` // apply: bắt đồng bộ, không trả về kết quả
`pref.edit().putInt("score", 100).commit()` // commit: đồng bộ, trả về boolean
// Đọc dữ liệu
`val score = pref.getInt("score", 0)` // 0 là giá trị mặc định nếu không tìm thấy key
- Bộ nhớ trong (Internal Storage):
 - Ghi: `openFileOutput("file.txt", Context.MODE_PRIVATE)`
 - Đọc: `openFileInput("file.txt")`
- Xử lý XML:
 - **SAX**: `while (parser.next() != XmlPullParser.END_DOCUMENT) { ... }` (Duyệt theo sự kiện).
 - **DOM**: `docBuilder.parse(inputStream) -> doc.getElementsByTagName("tag")` (Dựng cây đối tượng).



Checklist "Bẫy" tiểu tiết cho Mục 4:

- **Hardcoding**: Thắc hỏi "Tại sao phải dùng `strings.xml`", đáp án là để **Localization** (Đa ngôn ngữ) và **Maintainability** (Dễ bảo trì).
- **AssetManager**: Dùng để quản lý file trong thư mục `/assets`. Phải gọi `close()` sau khi đọc xong `InputStream`.

- **R.string.hello**: Kết quả trả về là một số nguyên (**int**), không phải là chuỗi **"Hello"**. Để lấy chuỗi phải dùng **getString(R.string.hello)**.
- **JSON vs XML**: JSON được ưu tiên hơn trong Mobile vì **nhẹ hơn** và **parse nhanh hơn** (Slide A13.1).
- Bổ sung sự khác biệt giữa **Mutable** và **Immutable** – đây là lợi thế lớn nhất của Kotlin so với Java trong việc quản lý dữ liệu tập hợp.

4.1 Resources & Localization

Resources là gì?

- Chứa strings, colors, dimens, drawables, layouts, styles/themes.
- Localization: tạo strings.xml theo locale (values-vi, values-en...).
- Tránh hardcode text; thiếu resource có thể gây lỗi hiển thị/crash (tùy trường hợp).
- Phân biệt /res và /assets:
 - **/res**: Được biên dịch, truy cập qua lớp **R** (ví dụ: **R.string.name**).
 - **/assets**: KHÔNG được biên dịch, truy cập qua đường dẫn tệp (path) bằng **AssetManager**.
- Các thư mục con trong /res cần nhớ:
 - **animator/**: XML cho property animations (mới).
 - **anim/**: XML cho view animations (cũ).
 - **raw/**: Lưu file thô như MP3, Video.
 - **xml/**: Các file XML tùy ý dùng trong thời gian chạy.

Resources & Localization (Cơ chế Qualifier):

- **Quy tắc tìm Resource (Qualifier)**: Android chọn tài nguyên dựa trên thư mục có hậu tố (Suffix).
 - Ví dụ: **values-en-rUS** (tiếng Anh Mỹ), **values-vi** (tiếng Việt).
 - **Bắt tiêu tiết**: Nếu máy để tiếng Pháp (**fr**) nhưng app chỉ có **values** (mặc định) và **values-vi**, hệ thống sẽ lấy ở thư mục **values** mặc định. Nếu **values** mặc định cũng không có -> **Crash App (ResourceNotFoundException)**.
- Phân biệt /res/raw và /assets (Câu hỏi kinh điển):

Tiêu chí	/res/raw	/assets
Truy cập	Qua ID lớp R (R.raw.filename)	Qua đường dẫn Path ("dir/file.txt")
Công cụ	getResources().openRawResource()	getAssets().open()

Cấu trúc	Không được tạo thư mục con	Được phép tạo cây thư mục con
Tên file	Chỉ chứa chữ thường, số, dấu gạch dưới	Tùy ý (giữ nguyên tên gốc)

- **Lớp R (R.java):** Là lớp chứa các hằng số `int` (ID) được hệ thống tự động sinh ra. Đề hay lừa: "Lớp R chứa nội dung của file ảnh" -> **Sai**, nó chỉ chứa **số định danh (ID)**.

4.2 Fonts, Files, SharedPreferences

Chọn nơi lưu nhanh:

Cần lưu	Giải pháp
Cấu hình nhỏ (flag, theme, token tạm...)	SharedPreferences (key-value)
Dữ liệu lớn/structure	File/DB (SQLite/Room/Realm)
Text/UI đa ngôn ngữ	Resources (strings.xml)

SharedPreferences – bẫy

- Phù hợp cho dữ liệu nhỏ; không dùng thay DB cho list lớn.
- Có thể commit/apply (khác nhau về đồng bộ).
- SharedPreferences (Bẫy đồng bộ):
 - **commit()**: Lưu dữ liệu **đồng bộ**, trả về **boolean** (thành công/thất bại). Chậm vì chặn thread.
 - **apply()**: Lưu dữ liệu **bất đồng bộ**, không trả về kết quả. Nhanh và được khuyến khích dùng hơn.

SharedPreferences (So sánh Behavior):

Đặc điểm	commit()	apply()
Cơ chế	Đồng bộ (Synchronous)	Bất đồng bộ (Asynchronous)
Trả về	boolean (Thành công/Thất bại)	void (Không trả về gì)

Hiệu năng	Chậm (Block Main Thread)	Nhanh (Ghi vào RAM ngay, ghi file sau)
Khuyến dùng	Khi cần biết kết quả ngay để xử lý	Ưu tiên dùng trong hầu hết trường hợp

Bẫy: "Dữ liệu SharedPreferences mất đi khi tắt app?" -> **Sai**, nó lưu vào file XML trong bộ nhớ máy, chỉ mất khi **Clear Data** hoặc **Uninstall app**.

4.3 Internal vs External Storage (Lưu trữ file)

- Internal Storage (Bộ nhớ trong):
 - Mặc định là riêng tư (Private). App khác không xem được.
 - Bị xóa khi gỡ app.
 - Không cần xin quyền (Permission) từ người dùng.
 - Hàm: `openFileOutput()`, `getFilesDir()`.
- External Storage (SD Card/Bộ nhớ dùng chung):
 - Có thể chia sẻ cho app khác.
 - Không bị xóa khi gỡ app (nếu lưu ở thư mục chung).
 - **Bắt buộc** xin quyền: `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`.

4.4 Fonts (API Level)

- **Thư mục:** `res/font` (Dùng cho Android 8.0+ / API 26+).
- **Cách dùng:** Truy cập qua `R.font.font_name`.
- **Bẫy:** Nếu dùng cho các đời Android cũ hơn, phải dùng thư viện hỗ trợ (Support Library) và khai báo trong XML theo cách khác.

4.5 XML Processing (SAX vs DOM)

- SAX (XmlPullParser - SAX Parser):
 - Cơ chế: Đọc theo luồng (Stream), gặp thẻ nào xử lý thẻ đó (`START_TAG`, `END_TAG`).
 - Ưu điểm: **Ít tốn RAM**, cực nhanh cho file lớn.
 - Nhược điểm: Chỉ đọc 1 chiều từ trên xuống, không quay lại được.
- DOM (Document Object Model):
 - Cơ chế: Đọc **toàn bộ** file rồi dựng thành cây đối tượng trong RAM.
 - Ưu điểm: Dễ truy xuất bất kỳ nút nào, dễ chỉnh sửa.
 - Nhược điểm: **Rất tốn RAM**, dễ gây lag nếu file XML lớn.

4.6 SQLite Types & SharedPrefs Path

Tiểu tiết về nơi lưu trữ thực tế:

- **SQLite Data Types:** SQLite chỉ có 5 kiểu dữ liệu: `NULL`, `INTEGER`, `REAL`, `TEXT`, `BLOB`. **Bấy:** Không có kiểu `BOOLEAN` (thường dùng 0/1) hay `DATETIME` (dùng String/Long).
- **SharedPreferences Path:** Thực chất là file `.xml` nằm tại `/data/data/<package_name>/shared_prefs/`.
- **SQLiteOpenHelper:** `onUpgrade` được kích hoạt khi biến `DATABASE_VERSION` trong code lớn hơn version hiện tại trên máy user.

Tiểu tiết về SQLite & Room:

- **Kiểu dữ liệu SQLite:** Chỉ có 5 kiểu: `NULL`, `INTEGER`, `REAL`, `TEXT`, `BLOB`. **Bấy:** Không có kiểu `BOOLEAN` (dùng 0/1) hay `DATETIME` (dùng String/Long).
- **Cursor.moveToFirst():** Hàm này trả về `Boolean` (có dòng đó hay không), không phải trả về dữ liệu.
- **Room Database:** Mặc định **cấm** chạy trên Main Thread (App sẽ crash ngay lập tức).

5) Data Persistence: XML/JSON – SQLite/Cursor – Room – Realm

[SYNTAX NHẬN DIỆN NHANH]

- **JSON Processing:**
 - `val obj = JSONObject(jsonString) -> Truy cập: obj.getString("name").`
 - `val array = JSONArray(jsonString) -> Truy cập: array.getJSONObject(0).`
- **SQLite & Cursor:**
 - Khởi tạo: `class MyDbHelper(context: Context) : SQLiteOpenHelper(context, DB_NAME, null, DB_VERSION)`
 - Thao tác: `db.insert("Table", null, contentValues) | db.query(...)` trả về `Cursor`.
 - Đọc Cursor: `if (cursor.moveToFirst()) { val data = cursor.getString(index) }`
- **Room (Phân biệt qua Annotation):**
 - **@Entity:** Khai báo bảng (Class).
 - **@Dao:** Interface chứa `@Query("SELECT...")`, `@Insert`, `@Delete`.
 - **@Database:** Lớp abstract kế thừa `RoomDatabase`.
 - **@Embedded:** Nhúng object vào bảng.
- **Realm (Object DB):**

- **Write:** `realm.writeBlocking { copyToRealm(studentObj) }`
- **Read:** `realm.query<Student>("name == $0", "An").find()`
- **SQLiteOpenHelper:** `onCreate(db: SQLiteDatabase)` và `onUpgrade(db, old, new)`.
- **Room Annotations (Cực quan trọng):** `* @Entity(tableName = "users")`
 - `@PrimaryKey(autoGenerate = true)`
 - `@Dao` (Interface chứa `@Query`, `@Insert`)
- **Realm CRUD:** `realm.writeBlocking { copyToRealm(student) }`.

Checklist "Bẫy" cho Mục 5 (Đọc kỹ trước khi chọn):

- **"Cursor.moveToFirst() trả về dữ liệu của dòng đầu tiên": Sai.** Nó trả về `boolean` (có dòng đó hay không), dữ liệu phải lấy qua `getString()`, `getInt()`.
- **"Room nhanh hơn SQLite thuần": Sai.** Room thực chất là "vỏ bọc" của SQLite, nó chỉ giúp viết code an toàn hơn (check lỗi SQL lúc compile), không làm tăng tốc độ xử lý thô.
- **"XML và JSON chỉ lưu được trên Internal Storage": Sai.** Chúng là định dạng dữ liệu, có thể lưu bất cứ đâu (Assets, Internal, External hoặc qua Network).
- **"Realm bắt buộc dùng SQL để truy vấn": Sai.** Realm dùng API hướng đối tượng (Query theo kiểu gọi hàm).

5.1 XML & JSON

Nhớ nhanh

- XML/JSON thường dùng để lưu cấu hình/dữ liệu dạng text hoặc trao đổi dữ liệu.
- Trắc nghiệm thường hỏi: format nào phổ biến cho API (JSON), format nào có tag (XML).
- JSON (JavaScript Object Notation):
 - Dựa trên ngôn ngữ JavaScript nhưng độc lập với nền tảng.
 - Gồm 2 cấu trúc: Cặp **Name/Value** (Object `{}`) và **Danh sách có thứ tự** (Array `[]`).
 - **Ưu điểm:** Nhẹ hơn, dễ đọc/ghi và dễ parse hơn XML.

Chi tiết XML: Phân biệt cơ chế **SAX (XmlPullParser)** - đọc theo luồng vs **DOM** - xây dựng cây đối tượng.

- **JSON:** Nhẹ, dễ parse, gồm Object `{ }` và Array `[ ]`.
- **XML Processing:**
 - **SAX (XmlPullParser):** Đọc theo luồng (Event-based), tốn ít bộ nhớ. Dùng `next()` để bắt `START_TAG`, `END_TAG`.
 - **DOM (W3C):** Xây dựng cây đối tượng (Tree-like), dễ truy xuất nhưng tốn RAM hơn.

Bây ở cơ chế đọc file (Parser):

- **`getString("key")`:** Nếu "key" không tồn tại $\rightarrow$ **Crash** (Quăng Exception).
- **`optString("key")`:** Nếu "key" không tồn tại $\rightarrow$ Trả về chuỗi rỗng `""` hoặc giá trị mặc định, **Không crash**.
- **Mẹo:** Cứ thấy chữ `opt` (Optional) là an toàn hơn chữ `get`.

Đặc điểm	SAX (XmlPullParser)	DOM (Document Object Model)
Cơ chế	Đọc theo luồng (Event-based)	Xây dựng cây (Tree-based)
Hành vi	Gọi <code>next()</code> để duyệt qua <code>START_TAG</code> , <code>TEXT</code> , <code>END_TAG</code> .	Load toàn bộ file vào RAM rồi mới truy xuất.
Bộ nhớ	Rất thấp (Phù hợp file lớn).	Cao (Dễ gây OutOfMemory nếu file lớn).
Truy xuất	Chỉ đọc 1 chiều từ trên xuống.	Truy xuất ngẫu nhiên (đi ngược, đi xuôi) thoải mái.

Snippet JSON cần nhớ: `JSONObject` (dùng `{ }`) và `JSONArray` (dùng `[ ]`).

Bẫy: "JSON khó đọc hơn XML" $\rightarrow$ **Sai**. JSON được ưa chuộng vì nhẹ và dễ đọc cho cả người và máy.

5.2 SQLite & Cursor

SQLite

- Embedded database (RDBMS) lưu trong file trên thiết bị.
- Query thường trả về Cursor để duyệt kết quả.

Hạn chế của Cursor (hay hời)

- Nhiều boilerplate; phải tự map dữ liệu sang object.
- Phải quản lý `close()` đúng để tránh memory leak.
- Query nặng chạy main thread gây lag/ANR (nên chạy background).
- **Quản lý thủ công:** Phải tự gọi `close()` để tránh rò rỉ bộ nhớ (memory leak).
- **Hiệu năng:** Việc duyệt từng dòng (row-by-row) chậm nếu tập kết quả lớn.
- **Boilerplate:** Phải ép kiểu (mapping) thủ công từ Cursor sang Object.
- **Vị trí:** Luôn phải nhớ `moveToFirst()` vì vị trí mặc định là `-1`.

Chi tiết SQLite: Thêm lưu ý về `Cursor` (luôn `moveToFirst`, bắt memory leak nếu không `close`).

`SQLiteOpenHelper` và trạng thái của `Cursor`:

- `SQLiteOpenHelper Lifecycle`:
 - `onCreate()`: Chỉ chạy **1 lần duy nhất** khi file database lần đầu được tạo. Dùng để tạo bảng (`CREATE TABLE`).
 - `onUpgrade()`: Chạy khi số `version` của DB tăng lên. Thường dùng `DROP TABLE` rồi gọi lại `onCreate()`.
- Hành vi của `Cursor`:
 - **Vị trí -1:** Sau khi query, `Cursor` trở vào trước dòng đầu tiên. Phải gọi `moveToFirst()` hoặc `moveToNext()` mới lấy được dữ liệu.
 - **Check dữ liệu:** `moveToNext()` trả về `true` nếu có dòng tiếp theo, `false` nếu hết dữ liệu.
 - **Bắt Memory Leak:** Nếu không gọi `cursor.close()`, file DB sẽ bị khóa và tốn RAM ngầm.
- **ContentValues:** Là một Map (`key-value`) dùng để đưa dữ liệu vào DB thông qua lệnh `insert()` hoặc `update()`. (Giúp tránh viết SQL thuần dài dòng).

Bắt Cursor: Mặc định ở vị trí `-1`, phải gọi `moveToFirst()`. Bắt buộc `close()` để tránh memory leak.

5.3 Room

Room (abstraction của SQLite)

- Entity – DAO – Database
- Compile-time check cho câu SQL; giảm boilerplate; hỗ trợ observable patterns (tuỳ môi trường).

Chi tiết Room: Bảng tra nhanh các Annotation quan trọng (`@Embedded`, `@Ignore`, `@ForeignKey`).

Room Annotations:

- **@Embedded**: Nhúng 1 object vào table (phẳng hóa cột).
- **@Ignore**: Bỏ qua trường không muốn lưu.
- **@ForeignKey**: Ràng buộc quan hệ giữa các bảng (CASCADE, RESTRICT).

Để phân biệt với SQLite thuần, hãy nhớ các từ khóa "Kiểm tra lúc biên dịch":

- 3 Thành phần bắt buộc:
 - **@Entity**: Khai báo cấu trúc bảng (Class).
 - **@Dao**: Interface chứa các câu lệnh SQL (**@Query**, **@Insert**, **@Delete**).
 - **@Database**: Lớp trừu tượng (Abstract) kết nối Entity và Dao.
- Annotation đặc biệt (Bấy tiểu tiết):
 - **@Embedded**: Dùng khi muốn nộp 1 Object vào trong 1 Entity (ví dụ: Object **Address** trong Entity **User**).
 - **@PrimaryKey(autoGenerate = true)**: Tự động tăng ID.
 - **@ForeignKey**: Tạo liên kết giữa 2 bảng. Thử nghiệm hay hỏi về **onDelete = CASCADE** (Xóa cha thì con mất theo).
- **Hành vi**: Mặc định Room **KHÔNG** cho phép thực thi query trên **Main Thread** (App sẽ crash).

5.4 Realm

Realm (nhận biết)

- Database hướng object; thao tác thuận tiện cho model object.
- Thử nghiệm thường hỏi so sánh mức khái niệm: Room/SQLite vs Realm.
- **Đặc điểm**: Không cần ORM, quản lý theo Object, hỗ trợ Offline hoàn hảo.
- **CRUD flow**: **realm.writeBlocking { copyToRealm(obj) }**.
- **Lưu ý**: Dữ liệu tự động cập nhật (live objects) và hỗ trợ Flow/Asynchronous Flow.

Chi tiết Realm: Thêm cú pháp CRUD đặc trưng (**writeBlocking**, **copyToRealm**, **findLatest**).

So sánh nhanh:

Tiêu chí	SQLite	Room	Realm
Mức thấp/cao	Thấp (raw SQL)	Cao (ORM-ish)	Cao (object DB)

Boilerplate	Nhiều	Ít hơn	Ít
Check SQL	Runtime	Compile-time (thường)	Không dùng SQL kiểu SQLite
Khi dùng	Nhỏ/tuỳ biến SQL	App Android hiện đại	Cần object DB nhanh

So sánh với SQLite:

- **Khác biệt cốt lõi:** Realm không dùng SQL. Nó lưu trực tiếp các Object.
- **Giao dịch (Transaction):** Mọi thao tác ghi (**Insert/Update/Delete**) **phải** nằm trong khối **writeBlocking { ... }**.
- **Live Objects:** Nếu bạn lấy ra 1 Object từ Realm, khi dữ liệu dưới DB thay đổi, Object đó **tự cập nhật** giá trị mới mà không cần query lại.
- **Snippet CRUD Realm:**
 - **realm.query<Student>().find():** Lấy danh sách.
 - **findLatest(it):** Tìm bản ghi mới nhất của đối tượng trong transaction.

6) Network & Cloud: REST – Firebase Auth – RTDB/Storage – Firestore compare

[SYNTAX NHẬN DIỆN NHANH]

- Retrofit (Interface):
 - **@GET("users/{id}") \$> @Path("id") id: Int** (Thay thế trực tiếp vào chuỗi URL).
 - **@GET("users") \$> @Query("sort") s: String** (Tạo URL dạng **?sort=abc**).
 - **@POST("users") \$> @Body user: User** (Gửi nguyên Object dưới dạng JSON).
- Firebase Auth:
 - **val user = FirebaseAuth.getInstance().currentUser** (Lấy user hiện tại, nếu **null** = chưa login).
 - **auth.signInWithEmailAndPassword(email, pass)** (Đăng nhập).

- Realtime Database (RTDB):
 - `val ref = FirebaseDatabase.getInstance().getReference("users")` (Trở tới node "users").
 - `ref.setValue(userObj)` (Ghi đè dữ liệu).
 - `ref.addValueEventListener` (Nghe liên tục) vs `ref.addListenerForSingleValueEvent` (Đọc 1 lần).
- Cloud Storage:
 - `storageRef.child("images/p.jpg").putFile(fileUri)` (Upload file).
 - `ref.getDownloadUrl().addOnSuccessListener { url -> ... }` (Lấy link để hiển thị ảnh).
- RTDB Listeners:
 - `addValueEventListener`: Nghe liên tục (Realtime).
 - `addListenerForSingleValueEvent`: Chỉ đọc 1 lần (Single).
- **Retrofit**: `@GET("users/{id}")` và `@Path("id")`.

Checklist "Bẫy" tiểu tiết cho Mục 6:

- **Internet Permission**: Đề hỏi "Tại sao App không gọi được API?", đáp án đầu tiên phải check là đã có `<uses-permission android:name="android.permission.INTERNET" />` trong Manifest chưa.
- **Main Thread**: Mặc định Android **cấm** gọi Network trên Main Thread (NetworkOnMainThreadException). Phải dùng `Thread`, `AsyncTask` hoặc các thư viện tự động xử lý ngầm như Retrofit.
- JSON Structure:
 - `{ }`: Trùng khớp với một Object trong Kotlin/Java.
 - `[ ]`: Trùng khớp với một List/ArrayList.
- **Firestore Rules**: Dữ liệu có thể bị lỗi "Permission Denied" nếu không cấu hình Security Rules trong Console (mặc định yêu cầu Auth).
- **Retrofit Annotations**: `@GET("path")`, `@POST("path")`, `@Body` (gửi data), `@Query` (lọc data trên URL). Chỉ cần nhìn Annotation là biết hành vi của hàm API.

6.1 Web services & REST

HTTP methods:

Method	Ý nghĩa
GET	Lấy dữ liệu
POST	Tạo mới / gửi data
PUT	Cập nhật (thường toàn phần)
PATCH	Cập nhật một phần
DELETE	Xoá

REST vs RPC (nhận biết)

- REST: dựa trên resource + HTTP methods.
- RPC: gọi hàm/remote procedure (SOAP/...); thường ít dùng trong app hiện đại.

So sánh REST vs SOAP/RPC: Thêm bảng phân biệt (WSDL, XML Envelope, Overhead).

- **REST:** Stateless, dùng URL, HTTP methods (GET, POST...). Dữ liệu: JSON/XML/Text.
- **SOAP:** Dùng WSDL để mô tả, XML Envelope để bao gói data, overhead cao.

Thư viện WebClient: Tóm tắt HttpURLConnection (thủ công) vs Retrofit (interface) vs Volley.

- **HttpURLConnection:** Thủ công, dùng `BufferedReader/StringBuilder` để đọc JSON.
- **Retrofit:** Chuyển API call thành method gọi hàm, tự parse JSON (Gson). Tốc độ nhanh nhất.
- **Volley:** Tốc độ trung bình, phù hợp cho nhiều request nhỏ đồng thời.

Bấy ở sự khác biệt giữa cách làm "thủ công" và dùng "thư viện":

- So sánh REST vs SOAP (Kiến thức kinh điển):

Tiêu chí	REST (Thịnh hành)	SOAP (Cổ điển/Doanh nghiệp)
----------	--------------------------	------------------------------------

Giao thức	HTTP, HTTPS, FTP, SMTP	Chủ yếu HTTP, SMTP
Định dạng	JSON (ưu tiên), XML, Text	Chỉ XML
Mô tả	Dựa trên tài nguyên (URL)	Dùng WSDL (file XML định nghĩa hàm)
Đóng gói	Không cần (Stateless)	Dùng XML Envelope (nặng/Overhead cao)

- So sánh các thư viện Networking:
 - **HttpURLConnection**: Thấp nhất (Low-level), phải tự mở kết nối, tự dùng `InputStreamReader` và `BufferedReader` để đọc từng dòng `InputStream` Code cực dài, dễ lỗi.
 - **Volley (Google)**: Quản lý theo hàng đợi (RequestQueue), hỗ trợ cache ảnh, phù hợp nhiều request nhỏ.
 - **Retrofit (Square)**: Dùng Interface để định nghĩa API. **Nhanh nhất**, tự động parse JSON sang Object (thông qua Gson).

6.2 Firebase Auth

Firebase Auth – ý chính

- Hỗ trợ nhiều kiểu đăng nhập (email/password, phone, federated...).
- Sau sign-in có token để verify; profile mở rộng thường lưu ở DB khác (RTDB/Firestore).
- **Hình thức**: Email/Password, Google, Facebook, Phone (SMS OTP),....
- **Code Keywords**: `FirebaseAuth.getInstance()`, `signInWithCredential`, `currentUser` (kiểm tra user đã login chưa).
- **Bắt**: Gói **Spark** (miễn phí) giới hạn 10k verification/tháng cho Phone Auth, gói **Blaze** mới tính phí vượt định mức.

Bắt hành vi & Giới hạn

- **Hành vi Check Login**: Để biết user đã đăng nhập hay chưa, dùng `auth.currentUser`. Nếu là `null` Chưa đăng nhập.
- **Bắt Token**: Sau khi đăng nhập thành công, Firebase trả về một **Token**. App gửi token này lên Server để xác thực chứ không gửi Password.
- **Giới hạn gói Spark (Slide A15.2)**:
 - Phone Auth: Miễn phí **10.000** xác thực/tháng. Vượt quá phải lên gói **Blaze** (\$0.01 - \$0.06/lần tùy quốc gia).
 - Các phương thức khác (Email, Google...): Miễn phí hoàn toàn (No-cost).

6.3 Realtime Database + Firestore

Realtime Database (RTDB)

- NoSQL dạng JSON tree; realtime sync; có offline support; security rules.

RTDB vs Firestore (nhận biết):

Tiêu chí	RTDB	Firestore
Model	1 JSON tree	Collection/Document
Query	Cơ bản	Mạnh hơn
Realtime	Có	Có
Scale	Tốt cho realtime đơn giản	Tốt cho app lớn + query phức tạp

- RTDB:
 - Cấu trúc: Một cây **JSON** khổng lồ.
 - Tính chất: Realtime (đồng bộ ngay lập tức), **Offline support** (lưu đệm khi mất mạng).
 - Listener: `addValueEventListener` (nghe thay đổi liên tục).
- Cloud Storage:
 - Dùng để lưu: Ảnh, Video, Audio.
 - Cơ chế: Dùng `StorageReference` trỏ đến file.
 - Action: `putFile()` (Upload), `getDownloadUrl()` (Lấy link để hiện ảnh bằng Glide), `delete()`.

Firebase Realtime Database (RTDB) vs Firestore

- **Mô hình RTDB:** Là một **Cây JSON duy nhất** (Single JSON Tree). Mọi dữ liệu lồng nhau.
- **Hành vi Offline:** RTDB tự động lưu dữ liệu vào bộ đệm (cache) khi mất mạng. Khi có mạng lại, nó tự **Sync** (đồng bộ) lên server.
- **Phân biệt Listeners (Rất hay hỏi):**
 - **`addValueEventListener`:** Nghe mọi thay đổi tại node đó và các node con. Chạy liên tục cho đến khi bị gỡ.
 - **`addListenerForSingleValueEvent`:** Chỉ đọc dữ liệu 1 lần duy nhất rồi tự ngắt. Phù hợp cho lấy cấu hình hoặc danh sách cố định.

- **Bắt query:** RTDB query rất yếu (chỉ lọc được trên 1 thuộc tính). Firestore query mạnh hơn (lọc trên nhiều thuộc tính cùng lúc).

6.4 Cloud Storage (Quản lý file)

Cloud Storage

- Lưu file (ảnh/video); thường đi kèm rule + link/metadata lưu ở DB.
- **Cách lấy dữ liệu:** Không lấy trực tiếp file, mà dùng `getDownloadUrl()` để lấy **đường dẫn (URL)**. Sau đó dùng thư viện **Glide** hoặc **Picasso** để hiển thị ảnh từ URL đó vào `ImageView`.
- **Lưu trữ:** Dữ liệu trong Storage là file thô (binary), còn link dẫn tới file đó thường được lưu trong RTDB để tiện quản lý.

7) Background & Notifications: Threading – Service – Broadcast – Local/FCM

[SYNTAX NHẬN DIỆN NHANH]

- **AsyncTask (Mô hình 3 tham số):** `* class MyTask : AsyncTask<Params, Progress, Result>()`
 - `doInBackground(vararg p: Params): Result ->` **Thread ngầm**, xử lý chính.
 - `onPostExecute(result: Result) ->` **UI Thread**, nhận kết quả để hiện lên màn hình.
- **Handler (Đưa việc về UI Thread):**
 - `Handler(Looper.getMainLooper()).post { // Cập nhật giao diện tại đây }`
- **Service Control:**
 - `startService(intent) | stopService(intent) | stopSelf()`
 - `onStartCommand(intent, flags, startId): Int ->` Trả về `START_STICKY, START_NOT_STICKY...`
- **Broadcast Receiver:**
 - Đăng ký code: `registerReceiver(receiver, IntentFilter("MY_ACTION"))`
 - Hủy: `unregisterReceiver(receiver)` (Thường gọi ở `onStop` hoặc `onDestroy`).
- **Notification (API 26+):**

- ```
// 1. Tạo Channel (Bắt buộc cho Android 8.0+) val
channel = NotificationChannel(ID, name, importance)
manager.createNotificationChannel(channel) // 2. Tạo
PendingIntent (Giấy ủy quyền) val pi =
PendingIntent.getActivity(ctx, 0, intent,
PendingIntent.FLAG_IMMUTABLE) // 3. Build & Notify
val builder = NotificationCompat.Builder(ctx,
ID).setSmallIcon(R.drawable.icon).setContentIntent(pi
) manager.notify(NOTIFY_ID, builder.build())
```
- `doInBackground(...)` : `Result` (Chạy ngầm).
  - `onPostExecute(result: Result)` (Về UI thread).

Checklist "Bẫy":

- **"Service chạy ở background nên không gây đứg máy": SAI.**  
Service vẫn chạy trên Main Thread, làm việc nặng vẫn gây ANR như thường.
- **"BroadcastReceiver dùng để hiển thị giao diện": SAI.** Nó không có UI, nó chỉ nhận sự kiện. Muốn hiện UI nó phải gọi Activity hoặc Notification.
- **"FCM Data Message luôn hiện thông báo": SAI.** Nó chỉ đưa dữ liệu về App, việc hiện thông báo hay không là do lập trình viên viết code trong `onMessageReceived`.
- **"onStartCommand chạy 1 lần duy nhất": SAI.** `onCreate` chạy 1 lần, nhưng `onStartCommand` chạy mỗi khi có lệnh `startService()`.
- **"Nút Back và Nút Home có cùng lifecycle": SAI.** Back = `onDestroy` (Huỷ), Home = `onStop` (Tạm dừng).

Service & Broadcast "Kịch độc":

- **Foreground Service:** Bắt buộc phải có một **Notification** không thể xóa (Persistent) để user biết app đang chạy ngầm (như Spotify/Google Maps).
- **Broadcast Receiver Time:** Hàm `onReceive` chỉ có tối đa **10 giây** để thực thi, quá thời gian này sẽ dính ANR.
- **Notification Rate Limit:** (Bạn đã có ý này nhưng nhấn mạnh thêm): Nếu update nhanh hơn **1 giây/lần**, hệ thống sẽ tự động drop (bỏ qua).

## 7.1 Multi-threading basics:

Thêm **LeakCanary**: Thư viện dùng để phát hiện rò rỉ bộ nhớ (Memory Leaks) trong quá trình phát triển.



Ngoài AsyncTask và Handler, còn một cách "bá đạo" để update UI từ thread ngầm: **activity.runOnUiThread { ... }**: Nếu snippet code có dòng này, nó sẽ đưa khối code bên trong về **Main Thread** để update UI. Đây là cách ngắn nhất thường thấy trong đề thi.

### Nguyên tắc vàng

- Không làm việc nặng (network/DB nặng) trên main thread -> dễ lag/ANR.
- Background thread xử lý, main thread chỉ update UI.

### Handler / post / message (mức khái niệm)

- Handler gắn với Looper (thường main looper).
- post(Runnable) hoặc sendMessage để đưa việc về thread của looper (thường là main) nhằm update UI.

### AsyncTask (mang tính lịch sử, hiểu ý tưởng)

- Dễ dùng cho tác vụ ngắn: doInBackground + onPostExecute update UI.
- Bẫy: dính lifecycle/leak nếu giữ reference Activity; không hợp tác vụ dài.

Bảng **AsyncTask**:

| Phương thức        | Thread chạy       | Chức năng                                                        |
|--------------------|-------------------|------------------------------------------------------------------|
| onPreExecute()     | UI Thread         | Chuẩn bị (ví dụ: hiện Loading)<br>Khởi tạo (hiện ProgressDialog) |
| doInBackground()   | Background Thread | Xử lý nặng (tải file, truy vấn DB)                               |
| onProgressUpdate() | UI Thread         | Cập nhật thanh tiến trình (%)                                    |
| onPostExecute()    | UI Thread         | Nhận kết quả từ background và hiển thị lên UI                    |

**Bẫy Dead-lock (từ đề cũ):** Xảy ra khi dùng **Handler-Post** mà trong đó lại thực hiện một thao tác chờ đợi vô hạn hoặc block chính UI Thread.

Multi-threading & AsyncTask (Tiểu tiết về Luồng):

- **ANR (Application Not Responding):** Xảy ra khi Main Thread bị block quá **5 giây**.
- **AsyncTask (Bẫy thứ tự & Thread):**
  - **onPreExecute:** Chạy đầu tiên (UI Thread).
  - **doInBackground:** Chạy việc nặng (Background Thread). Bẫy: Không được cập nhật UI trực tiếp ở đây. Phải gọi **publishProgress()**.
  - **onProgressUpdate:** Chạy sau khi gọi **publishProgress** (UI Thread).
  - **onPostExecute:** Chạy cuối cùng sau khi xong việc (UI Thread).
- **Handler & Looper:** **Handler** là người đưa thư, **Looper** là người vận chuyển. Nếu khởi tạo **Handler()** không tham số, nó mặc định gắn với Looper của Thread hiện tại. Muốn update UI từ Thread ngầm, phải dùng **Handler(Looper.getMainLooper())**.

## 7.2 Services

### Service – nhớ nhanh

- Lifecycle: onCreate → onStartCommand/onStart → onDestroy
- Service chạy nền nhưng không tự có thread; nếu làm nặng phải tạo thread/async.

Bổ sung ý tưởng về **Battery Saver** (đề 2018):

- **Tốn pin nhất:** Wifi, GPS, Bluetooth, Màn hình sáng, CPU chạy tác vụ nặng ngầm.
- **Kỹ thuật bắt tình trạng pin:** Dùng **BroadcastReceiver** lắng nghe Action **ACTION\_BATTERY\_CHANGED**.
- **Chạy khi khởi động lại máy:** Khai báo quyền **RECEIVE\_BOOT\_COMPLETED** trong Manifest và dùng **BroadcastReceiver** bắt sự kiện **BOOT\_COMPLETED**.

Bẫy hành vi "Sống/Chết":

- onStartCommand return values:
  - **START\_STICKY:** Hệ thống kill vì thiếu RAM -> Tự khởi động lại nhưng Intent = **null**. (Dùng cho trình phát nhạc).
  - **START\_NOT\_STICKY:** Hệ thống kill -> Không khởi động lại cho đến khi có Intent mới.
  - **START\_REDELIVER\_INTENT:** Tự khởi động lại và **gửi lại Intent cũ**. (Dùng cho download file).
- Bẫy Battery Saver (Slide A16):
  - Khi ở chế độ tiết kiệm pin, hệ thống sẽ hạn chế Service ngầm.

- Cách bắt sự kiện pin: Dùng `Intent.ACTION_BATTERY_CHANGED`.
- **Tiểu tiết:** Quyền `RECEIVE_BOOT_COMPLETED` cho phép app tự chạy lại sau khi reboot.

## 7.3 BroadcastReceiver

### Receiver – keywords

- `onReceive(Context, Intent)`
- Đăng ký: static (Manifest) hoặc dynamic (`registerReceiver`)
- Normal broadcast vs Ordered broadcast (priority/abort)

Quy định API Level:

- **Bắt API 26 (Android 8.0):** Hầu hết các Implicit Broadcast (như `ACTION_PACKAGE_ADDED`) không còn được phép đăng ký trong Manifest (Static). Phải đăng ký trong Code (Dynamic).
- **Thời hạn xử lý:** `onReceive()` chỉ có khoảng **10 giây** để thực thi. Nếu lâu hơn -> ANR.
- **Ordered Broadcast:** Cho phép các Receiver nhận theo thứ tự **priority**. Có thể dùng `abortBroadcast()` để ngăn các App phía sau nhận được tin nhắn này.

## 7.4 Notifications

### Local Notification

- Có icon/title/text; tap có thể mở Activity (`PendingIntent` tùy bài).
- Android 8.0+ (API 26+) phải tạo Notification Channel.

### FCM (Remote Notification) – flow

- Device đăng ký -> nhận registration token
- Server gửi message tới FCM backend
- FCM chuyển tới thiết bị
- Thiết bị nhận: notification message hoặc data message

**Notification message vs Data message:**

| Loại         | Đặc điểm                                         |
|--------------|--------------------------------------------------|
| Notification | Hệ thống có thể tự hiển thị (tùy trạng thái app) |
| Data         | Key-value; app tự xử lý (thường linh hoạt hơn)   |

## Local Notification (Bổ sung sâu)

- **Thành phần:** Icon (bắt buộc), Title, Content text.
- **Notification Channel (API 26+):** Phải tạo channel mới hiển thị được thông báo.
- **PendingIntent:** Là "giấy ủy quyền" để hệ thống mở Activity khi user click vào notification.
- **Quản lý:** `setAutoCancel(true)` (click xong tự biến mất), `cancel(ID)` (xóa 1 cái), `cancelAll()` (dọn sạch).

Quy tắc hiển thị:

- PendingIntent (Flags):
  - **FLAG\_UPDATE\_CURRENT:** Nếu Intent đã tồn tại, giữ nguyên nhưng cập nhật dữ liệu mới (Extra).
  - **FLAG\_IMMUTABLE:** Không cho phép thay đổi Intent sau khi tạo (Yêu cầu bắt buộc từ Android 12).
- **Bắt Notification Channel:** Nếu App nhắm đến API 26+ mà không tạo Channel, lệnh `notify()` sẽ chạy nhưng **không có gì hiện ra**.
- **Rate Limit (Slide A15.4):** Nếu cập nhật thông báo liên tục (nhiều lần trong < 1 giây), hệ thống sẽ **drop (bỏ qua)** các bản cập nhật đó.

## 7.5 Firebase Cloud Messaging (FCM - Remote)

- **Registration Token:** Mã định danh duy nhất cho mỗi máy. Dùng để gửi đúng người.
- **Payload:** Tối đa 4KB.
- **Phân biệt loại tin nhắn:**
  - **Notification Message:** Hệ thống tự hiển thị khi app chạy nền.
  - **Data Message:** App tự xử lý trong `onMessageReceived` (linh hoạt hơn).

Behavior:

- Hành vi nhận tin (Khi App ở Background):
  - **Notification Message:** Hệ thống tự hiển thị lên Tray (thanh thông báo). Khi user click mới vào App. Code trong `onMessageReceived` **không chạy**.
  - **Data Message:** App tự xử lý code trong `onMessageReceived` bất kể app đang ở foreground hay background.

- **Registration Token:** Không cố định mãi mãi. Nó sẽ bị thay đổi khi: App được cài lại, xóa dữ liệu app, hoặc token cũ hết hạn.
- **Payload:** Giới hạn cứng **4KB**.

## 7.6 Foreground Service & AsyncTask Cancel

- **Foreground Service:** Khác với Background Service, nó **bắt buộc** phải hiển thị một Notification không thể xóa (Persistent Notification) để user biết app đang chạy ngầm (ví dụ: Google Maps, Spotify).
- **AsyncTask Cancel:** Khi gọi `cancel(true)`, hàm `doInBackground` vẫn tiếp tục chạy cho đến khi xong hoặc bạn phải check `isCancelled()` bên trong vòng lặp. Nó không dừng ngay lập tức một cách thần kỳ.

# Trang cuối: Mini checklist trước khi đi thi

## 10 điểm chốt

- 1) **API 26+ (Oreo)**: Bắt buộc có **Notification Channel**. Không có channel = code không lỗi nhưng không hiện thông báo.
- 2) **Service** không tự tạo thread, Chạy trên **Main Thread**. Tác vụ nặng phải tự tạo Thread/Coroutine.
- 3) **savedInstanceState**: **null** khi khởi tạo lần đầu; **not-null** khi xoay màn hình hoặc hệ thống kill rồi khôi phục.
- 4) **BACK vs HOME**: BACK gọi **onDestroy** (huỷ); HOME chỉ tới **onStop** (dừng, vẫn còn trong RAM).
- 5) **FragmentManager**: add/replace/commit (+ backstack). Cần **commit()** để thực thi. Dùng **addToBackStack()** để nút BACK không thoát app mà quay lại fragment trước.
- 6) **Permission**: Khai báo Manifest là "xin", gọi Runtime Permission (Android 6+) là "được cho".
- 7) **Adapter**: ListView dùng **getView()**. RecyclerView dùng **onBindViewHolder()** để gán data.
- 8) **Room**: Entity (Bảng), DAO (Lệnh SQL), Database (Kết nối). Check lỗi SQL ngay khi **Biên dịch (Compile-time)**.
- 9) **RTDB vs Firestore**: RTDB là 1 cây JSON khổng lồ; Firestore là các Collection/Document query mạnh hơn.
- 10) **Threading**: **doInBackground** (việc nặng ngầm); **onPostExecute/runOnUiThread** (cập nhật UI).

# PHỤ LỤC A — TRẮC NGHIỆM “LẠ” / TRIVIA (KIỂU: LỊCH SỬ, AI SÁNG TẠO, NĂM NÀO)

Checklist "Bẫy" từ Phụ lục A:

- **Bẫy Timeline:** Đừng nhầm năm Google mua Android (2005) với năm ra mắt máy đầu tiên (2008).
- **Bẫy IDE:** Nếu hỏi IDE dựa trên IntelliJ IDEA -> Chọn Android Studio. Nếu hỏi IDE cũ -> Chọn Eclipse ADT.
- **Bẫy Runtime:** Nhớ mốc **Android 5.0** là bước ngoặt từ Dalvik sang ART (AOT).
- **Bẫy Figma:** Figma KHÔNG phải app native để cài code, nó là công cụ thiết kế chạy trên trình duyệt (browser-based).

Resource Qualifiers Priority:

- **Thứ tự ưu tiên (simplified):** MCC/MNC -> Language (**values-vi**) -> Layout Direction -> Region -> Screen Size -> Screen Orientation (**layout-land**) -> Screen Density (**drawable-xhdpi**).
- **Bẫy:** Nếu máy đang ở chế độ Landscape nhưng thư mục **layout-land** không có file đó, nó sẽ quay về tìm ở thư mục **layout** mặc định.

Mục tiêu: nếu thầy ra kiểu “ai tạo ra / cái này thuộc ai / cái nào xuất hiện trước” thì có 1 góc để khoanh nhanh.

## Nhóm 1: Nguồn gốc & Các cột mốc thời gian (Dates & Milestones)

| STT | Chủ đề                      | Chi tiết tiểu tiết (Cần nhớ để khoanh)                                           |
|-----|-----------------------------|----------------------------------------------------------------------------------|
| 1   | Android Inc.                | Thành lập <b>10/2003</b> bởi Andy Rubin, Rich Miner, Nick Sears, và Chris White. |
| 2   | Google & Android            | Google mua lại Android vào <b>tháng 7/2005</b> .                                 |
| 3   | OHA (Open Handset Alliance) | Thành lập <b>05/11/2007</b> (Liên minh gồm Google, Samsung, Intel, Sony...)      |
| 4   | Thiết bị đầu tiên           | <b>T-Mobile G1</b> (hay HTC Dream) ra mắt <b>22/10/2008</b> .                    |
| 5   | Tổ chức                     | Android thuộc về Google, dựa trên <b>nhân Linux</b> (Linux kernel).              |

## Nhóm 2: "Cha đẻ" & Những nhân vật tiêu biểu

| STT | Nhân vật            | Gắn liền với khái niệm / Công nghệ                         |
|-----|---------------------|------------------------------------------------------------|
| 1   | Andy Rubin          | "Cha đẻ" Android (Sáng lập Android Inc.)                   |
| 2   | James Gosling       | Người tạo ra ngôn ngữ <b>Java</b> (Sun Microsystems).      |
| 3   | Linus Torvalds      | Người khởi xướng nhân <b>Linux</b> .                       |
| 4   | <b>Roy Fielding</b> | Tác giả khái niệm/luận án về <b>REST</b> .                 |
| 5   | Trygve Reenskaug    | Người khởi nguồn mô hình <b>MVC</b> (kỷ nguyên Smalltalk). |

### Nhóm 3: Công cụ & Ngôn ngữ (Tools & Languages)

| STT | Công cụ / Ngôn ngữ                | Đặc điểm hành vi                                                                 |
|-----|-----------------------------------|----------------------------------------------------------------------------------|
| 1   | <b>Android Studio</b>             | IDE chính thức hiện nay; dựa trên <b>IntelliJ IDEA</b> (JetBrains).              |
| 2   | Eclipse ADT                       | IDE phổ biến <b>trước khi</b> có Android Studio.                                 |
| 3   | <b>Kotlin (Nguồn gốc)</b>         | Do JetBrains phát triển (tên một hòn đảo ở Nga). Xuất hiện lần đầu <b>2011</b> . |
| 4   | <b>Kotlin-first</b>               | Được Google công bố là ngôn ngữ ưu tiên tại Google I/O <b>2019</b> .             |
| 5   | Figma                             | Công cụ thiết kế UI cộng tác, <b>browser-based</b> (chạy trên trình duyệt).      |
| 6   | Prototyping                       | Tính năng Figma dùng để mô phỏng hành vi app thực tế qua các màn hình.           |
| 7   | Android app được build ra file gì | file APK (Android Package Kit) hoặc AAB (Android App Bundle)                     |
| 8   | Gradle là gì                      | build system                                                                     |
| 9   | ADB dùng làm gì                   | debug/install/logcat                                                             |

### Nhóm 4: Cơ chế thực thi & Đồ họa (Runtime & UI Concepts)

| STT | Khái niệm               | So sánh & Hành vi                                                              |
|-----|-------------------------|--------------------------------------------------------------------------------|
| 1   | <b>Dalvik (Runtime)</b> | Cơ chế cũ. Dùng <b>JIT (Just-In-Time)</b> để biên dịch.                        |
| 2   | ART (Runtime)           | Mặc định từ <b>Android 5.0 (Lollipop)</b> . Dùng <b>AOT (Ahead-Of-Time)</b> .  |
| 3   | Cơ chế ART vs Dalvik    | ART giúp app mở nhanh hơn, mượt hơn nhờ biên dịch trước khi chạy.              |
| 4   | <b>9-patch Image</b>    | File có đuôi <b>.9.png</b> , giúp co giãn ảnh (background/button) không bị vỡ. |



|   |          |                                                                                      |
|---|----------|--------------------------------------------------------------------------------------|
| 5 | Firebase | Được Google mua lại ( <b>2014</b> ); là nền tảng <b>Backend-as-a-Service</b> (BaaS). |
|---|----------|--------------------------------------------------------------------------------------|

- **SQLite: database nhúng (embedded), chạy trong app (không cần server).**
- **JSON vs XML: JSON nhẹ hơn, phổ biến cho REST API. (bạn đã có, nhưng thêm 1 câu “why”).**
- **IDE/Build: Android Studio (IntelliJ), Gradle build.**
- **MVC/MVVM: Activity/Fragment thường là “Controller/View layer”.**

#### **BÃY HAY GẬP (đọc 30 giây trước khi nộp):**

- Câu “AI/AI SÁNG TẠO/THUỘC VỀ” thường chỉ cần nhớ 1–2 tên nổi bật (Rubin, Gosling, Torvalds, Fielding, Reenskaug).
- Câu về “từ phiên bản nào” hay nhầm: Notification Channel (API 26+), ART mặc định (Android 5.0+).
- Nếu hỏi về IDE: Android Studio (IntelliJ base) vs Eclipse ADT (cũ).
- **Service vs Thread:** Đừng nhầm Service là Thread. Service chỉ là một component không giao diện, nó vẫn có thể làm được máy nếu không tạo Thread ngầm.
- **Notification Rate Limit:** Nếu cập nhật thông báo quá nhanh (< 1 giây), hệ thống có thể drop bớt các update.
- **Firebase Offline:** RTDB tự có offline, nhưng phải cấu hình trong code để dữ liệu tồn tại sau khi tắt hẳn app (`setPersistenceEnabled(true)`).
- **Implicit Intent:** Mở Camera, gọi điện, gửi mail. Nếu không có app nào đáp ứng được Intent này, app sẽ bị **crash**.
- **"Service giúp app chạy nhanh hơn vì nó chạy process riêng": SAI.** Service chạy cùng process và cùng Main Thread với App.
- **"Có thể hiển thị Toast từ một Background Thread": SAI.** Toast cần UI Context, phải hiện từ Main Thread.
- **"9-patch image (.9.png) giúp ảnh nét hơn": SAI.** Nó chỉ giúp ảnh co giãn mà không vỡ ở các vùng chỉ định.
- **"Firebase Realtime DB là một SQL Database": SAI.** Nó là **NoSQL** dạng JSON Tree.
- **"Chỉ cần khai báo INTERNET permission là đủ gọi API": SAI.** Nếu gọi HTTPS mà certificate lỗi hoặc chạy trên Main Thread vẫn crash.

#### **ANDROID VERSION (hay bị hỏi “tên bánh / phiên bản”):**

- 1.5 Cupcake
- 1.6 Donut
- 2.0/2.1 Eclair
- 2.2 Froyo
- 2.3 Gingerbread
- 3.0 Honeycomb (Dành riêng cho Tablet)
- 4.0 Ice Cream Sandwich
- 4.1–4.3 Jelly Bean
- 4.4 KitKat

- 5.0 Lollipop (**ART** chính thức)
- 6.0 Marshmallow (**Runtime Permission**)
- 7.0 Nougat
- 8.0 Oreo (**Notification Channel**)
- 9 Pie
- 10+ dùng số (10, 11, 12, 13, 14...)

### TRIVIA “NGÔN NGỮ LẬP TRÌNH” (nếu thầy lỡ ra kiểu kỹ thuật lập trình):

- **C**: Dennis Ritchie
- **C++**: Bjarne Stroustrup
- **Java**: James Gosling
- **C#**: Anders Hejlsberg
- **OOP (khởi nguồn)**: Simula (Ole-Johan Dahl & Kristen Nygaard)

## FIX KOTLIN

- **"Kotlin Type Philosophy**: Mọi kiểu dữ liệu (Int, Boolean, Char...) đều là **Đối tượng (Non-Primitive)**. Không dùng kiểu viết thường như java (int, long)."
- **String Template**: `$var` (biến đơn), `${expr}` (biểu thức).
- **Collection**: `listOf` (Read-only), `mutableListOf` (Read-write).
- **Boolean strictly**: `true/false` (không dùng 0/1).
- **Data Class**: Tự động tạo `toString`, `equals`, `hashCode`, `copy`.
- **Sealed Class**: Giới hạn kế thừa trong cùng 1 file, cực mạnh khi đi kèm `when`.
- **Delegation**: Từ khóa `by` (Ủy quyền), hay dùng nhất là `by lazy` (chỉ khởi tạo khi cần dùng đến).
- **Equality**:
  - `==` : Gọi hàm `.equals()`.
  - `===` : Kiểm tra xem có cùng trỏ vào 1 ô nhớ không.
- Coroutines:
  - Dùng `suspend fun` cho các tác vụ tốn thời gian.
  - `delay()` không gây block thread (khác với `Thread.sleep()`).
- **Interface**: Khác Java cũ, Interface trong Kotlin **được phép** có code xử lý bên trong hàm.
- **Static Context**: Dùng `companion object` (vì Kotlin không có `static`).
- **Inheritance**: Class mặc định bị khóa (`final`), phải thêm `open class` thì mới cho phép kế thừa.
- **Collections**: Phân biệt rõ `listOf` (không sửa được) và `mutableListOf` (sửa được).
- **Scope Functions**: Ghi nhớ 5 hàm: `let`, `run`, `with`, `apply`, `also` để xử lý object trong phạm vi hẹp.
- **Functional Programming**: Dùng `filter`, `map`, `reduce` để xử lý list gọn nhẹ.

- **Extension Functions:** Thêm hàm mới vào Class có sẵn mà không cần kế thừa. Cú pháp: `fun ClassName.functionName()`.
- **Type Equality:** Không so sánh trực tiếp `Char` và `Int` bằng `==`.
- **SAM Interface:** Dùng `fun interface` cho các giao diện có 1 hàm duy nhất.
- **Return Type:** Hàm không trả về giá trị thì mặc định trả về `Unit`.
- **Type Check:** Dùng `is` để kiểm tra kiểu và tận dụng **Smart Casting**.

## NHÓM 1: CƠ CHẾ HỆ THỐNG & KIẾN TRÚC

| Câu hỏi                                        | Đáp án bổ sung cho Cheatsheet                                                                                                                             |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hệ điều hành cô lập ứng dụng bằng cách nào?    | Gán cho mỗi ứng dụng một <b>ID người dùng Linux (UID) duy nhất</b> và thiết lập quyền truy cập tệp tương ứng (Sandbox).                                   |
| Hai ứng dụng chạy chung tiến trình khi nào?    | Khi cả hai được gán cùng một <b>sharedUserId</b> trong Manifest và được ký bằng <b>cùng một chứng chỉ (certificate)</b> .                                 |
| Cơ chế Controller yêu cầu Model/View thay đổi? | Thông qua các phương thức <b>callback</b> và các <b>listener</b> đã được đăng ký (Observer Pattern).                                                      |
| Thuật ngữ "headless fragment" là gì?           | Là một Fragment <b>không có giao diện người dùng</b> (không ghi đè <code>onCreateView()</code> hoặc trả về <code>null</code> ), dùng để xử lý tác vụ nền. |

## NHÓM 2: LAYOUT & CONSTRAINTLAYOUT

| Câu hỏi                              | Đáp án bổ sung cho Cheatsheet                                                |
|--------------------------------------|------------------------------------------------------------------------------|
| Thiếu vertical constraint ở runtime? | View sẽ nhảy về tọa độ <b>[0,0]</b> (góc trên cùng bên trái của layout cha). |

|                                       |                                                                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Điều kiện thiết lập Aspect Ratio?     | Ít nhất một chiều ( <b>width</b> hoặc <b>height</b> ) phải được đặt là <b>0dp</b> ( <b>MATCH_CONSTRAINT</b> ).               |
| Tính năng 'Autoconnect' là gì?        | Tự động tạo các ràng buộc cho View đến layout cha khi bạn kéo thả chúng trong Layout Editor.                                 |
| Cách thêm thư viện ConstraintLayout?  | Thêm dependency:<br><b>implementation("androidx.constraintlayout:constraintlayout:2.x.x")</b><br>trong <b>build.gradle</b> . |
| ListView/GridView thuộc danh mục nào? | Nằm trong danh mục <b>Legacy</b> (Di sản) của bảng Palette trong Android Studio.                                             |



### NHÓM 3: ADAPTER & RECYCLERVIEW (RV)

| Câu hỏi                                     | Đáp án bổ sung cho Cheatsheet                                                                                 |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Phương thức ghi đè khi dùng custom objects? | Ghi đè phương thức <b>toString()</b> trong lớp đối tượng hoặc tùy chỉnh trong <b>getView()</b> của Adapter.   |
| ArrayAdapter với custom object cần làm gì?  | Phải ghi đè <b>toString()</b> để hiển thị text mặc định hoặc tạo Custom Adapter kế thừa <b>ArrayAdapter</b> . |
| Cách thêm đường kẻ phân cách trong RV?      | Sử dụng lớp <b>DividerItemDecoration</b> và thêm vào RV qua <b>addItemDecoration()</b> .                      |
| Thành phần xử lý hoạt ảnh (animations)?     | Là <b>ItemAnimator</b> (mặc định là <b>DefaultItemAnimator</b> ).                                             |

|                                                     |                                                                                                                                      |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Widget con trực tiếp của <b>colorsHScrollView</b> ? | Thường là một <b>ViewGroup</b> duy nhất (như <b>LinearLayout</b> hoặc <b>ConstraintLayout</b> ) vì <b>ScrollView</b> chỉ chứa 1 con. |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|

## NHÓM 4: STORAGE & RESOURCES

| Câu hỏi                                                      | Đáp án bổ sung cho Cheatsheet                                                                                                         |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Thư mục lưu launcher icon?                                   | Thư mục <b>res/mipmap/</b> (để tối ưu độ phân giải biểu tượng trên các màn hình khác nhau).                                           |
| Khác biệt <b>SharedPreferences</b> & <b>getPreferences</b> ? | <b>getSharedPreferences()</b> dùng cho toàn app (cần tên file); <b>getPreferences()</b> dùng riêng cho Activity (không cần tên file). |
| Phương thức đọc mảng chuỗi trong Kotlin?                     | Sử dụng: <b>resources.getStringArray(R.array.ten_mang)</b> .                                                                          |

## NHÓM 5: SERIALIZATION & REALM

| Câu hỏi                                   | Đáp án bổ sung cho Cheatsheet                                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Giải tuần tự hóa (Deserialization) là gì? | Là quá trình chuyển đổi một chuỗi dữ liệu (JSON) ngược lại thành một <b>đối tượng Kotlin/Java</b> . |
| Plugin cho Kotlin Serialization?          | <b>id("org.jetbrains.kotlin.plugin.serialization")</b> trong <b>build.gradle</b> (Project level).   |
| Annotation <b>@SerializedName</b> ?       | Dùng để chỉ định <b>tên Key trong JSON</b> nếu nó khác với tên biến trong class Kotlin.             |

|                                       |                                                                                                                      |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Tham số <b>ignoreUnknownKeys?</b>     | Nếu <b>true</b> , app sẽ bỏ qua các key lạ trong JSON mà class Kotlin không khai báo (tránh lỗi).                    |
| Phương thức <b>encodeToString?</b>    | Chuyển đổi một đối tượng thành một chuỗi JSON.                                                                       |
| Mục đích của <b>@Serializable?</b>    | Đánh dấu class để compiler tạo mã hỗ trợ tuần tự hóa tự động.                                                        |
| Vị trí thêm plugin Realm?             | Plugin ID thêm ở <b>Project-level build.gradle</b> ; apply plugin ở <b>Module-level build.gradle</b> .               |
| Class Kotlin cần gì để lưu vào Realm? | Phải kế thừa (extend) từ lớp <b>RealmObject</b> .                                                                    |
| Định nghĩa schema trong Realm?        | Truyền vào một <b>Set các class</b> (ví dụ: <b>schema = setOf(Person::class)</b> ) trong <b>RealmConfiguration</b> . |
| Phương thức nhận kết quả dạng Flow?   | Sử dụng hàm <b>.asFlow()</b> để nhận các cập nhật dữ liệu liên tục.                                                  |



## NHÓM 6: WEBVIEW & JAVASCRIPT

| Câu hỏi                                       | Đáp án bổ sung cho Cheatsheet                                                             |
|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| Mục đích của <b>shouldOverrideUrlLoading?</b> | Cho phép app kiểm soát việc mở liên kết (mở ngay trong WebView hay mở trình duyệt ngoài). |
| Di chuyển theo số bước cụ thể?                | Sử dụng phương thức <b>goBackOrForward(int steps)</b> (số dương đi tới, số âm đi lùi).    |

|                                   |                                                                                                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------|
| Câu lệnh thiết lập cầu nối?       | <code>webView.addJavascriptInterface(MyInterface(), "AndroidBridge").</code>                        |
| HtmlAndroidBridge trong JS là gì? | Là <b>tên định danh (alias)</b> của đối tượng Kotlin được phơi bày ra để JS có thể gọi.             |
| Quyền (permission) đọc danh bạ?   | <code>&lt;uses-permission android:name="android.permission.READ_CONTACTS" /&gt;.</code>             |
| Cách Java gọi hàm JavaScript?     | <code>webView.loadUrl("javascript:updateCounter()")</code> hoặc <code>evaluateJavascript()</code> . |
| Tham số Alias có vai trò gì?      | Tạo ra một <b>biến toàn cục trong môi trường JavaScript</b> để JS truy cập vào code Native.         |



## NHÓM 7: KHÁC (XML, FIREBASE)

| Câu hỏi                                | Đáp án bổ sung cho Cheatsheet                                                                              |
|----------------------------------------|------------------------------------------------------------------------------------------------------------|
| Khác biệt XML và HTML?                 | XML dùng để <b>vận chuyển/lưu trữ dữ liệu</b> ; HTML dùng để <b>hiển thị dữ liệu</b> .                     |
| Đặc tính "Transparency" của XML?       | Là tính <b>minh bạch/dễ đọc</b> , giúp con người có thể hiểu nội dung mà không cần công cụ đặc biệt.       |
| URI chuyển hướng Facebook dán vào đâu? | Dán vào mục <b>"Valid OAuth Redirect URIs"</b> trong phần Facebook Login Settings trên Facebook Developer. |
| Mục đích của Firebase BoM?             | Giúp <b>quản lý phiên bản đồng nhất</b> cho tất cả các thư viện Firebase trong dự án, tránh xung đột.      |

